GUÍA PRÁCTICA · GIT

Conflictos de Git

Cómo entenderlos y resolverlos sin miedo.
10 ejercicios reales, explicados paso a paso.

10/10	57	0	51
ejercicios	comprobaciones	fallos	capturas reales

LAS CINCO MIRADAS

1. **Detectar** — estado y archivos afectados
2. **Entender** — qué quiso cada lado
3. **Resolver** — una decisión por bloque
4. **Verificar** — sin marcas y con pruebas
5. **Prevenir** — ramas cortas, integrar seguido

10 ejercicios paso a paso · Mapa mental en Canvas HTML5 · 51 capturas reales

Un conflicto no es un error: es una pausa para decidir.

Ivan · Octubre 2026 · Windows 11

github.com/75749748-blip/conflictos-git-ejercicios

ANTES DE EMPEZAR

Tres ideas que lo cambian todo

No hace falta memorizar comandos. Hace falta entender qué te está preguntando Git.

01

Un conflicto no es un error

Git detuvo la operación para no perder trabajo. El repositorio sigue intacto; solo está esperando una decisión.

02

Git no elige por ti

Muestra las dos versiones y unas marcas. Elegir cuál se queda es una decisión humana, no un comando mágico.

03

Resolver es decidir

Puedes conservar un lado, combinar los dos o eliminar el archivo. Lo importante es que el resultado tenga sentido.

LAS CUATRO PALABRAS QUE VERÁS

Rama

Una línea de trabajo paralela. `main` suele ser la principal.

Merge

Une dos ramas en un commit nuevo que conserva las dos historias.

Marcador

`<<<<<<<`, `=======` y `>>>>>>>` separan las versiones en conflicto.

Índice

La zona donde preparas lo que irá en el próximo commit con `git add`.

Lo que necesitas: Git instalado, una terminal, un editor de texto y este documento. Nada más.

```
<<<<<<< HEAD
linea 2 (main)
=======
linea 2 (feature)
>>>>>>> feature
```

Cómo leer las marcas

- `HEAD` es tu rama actual.
- `=======` separa las dos versiones.
- `>>>>>>> feature` nombra la rama que llega.

No borres las líneas sin leerlas: primero decide, después limpia.

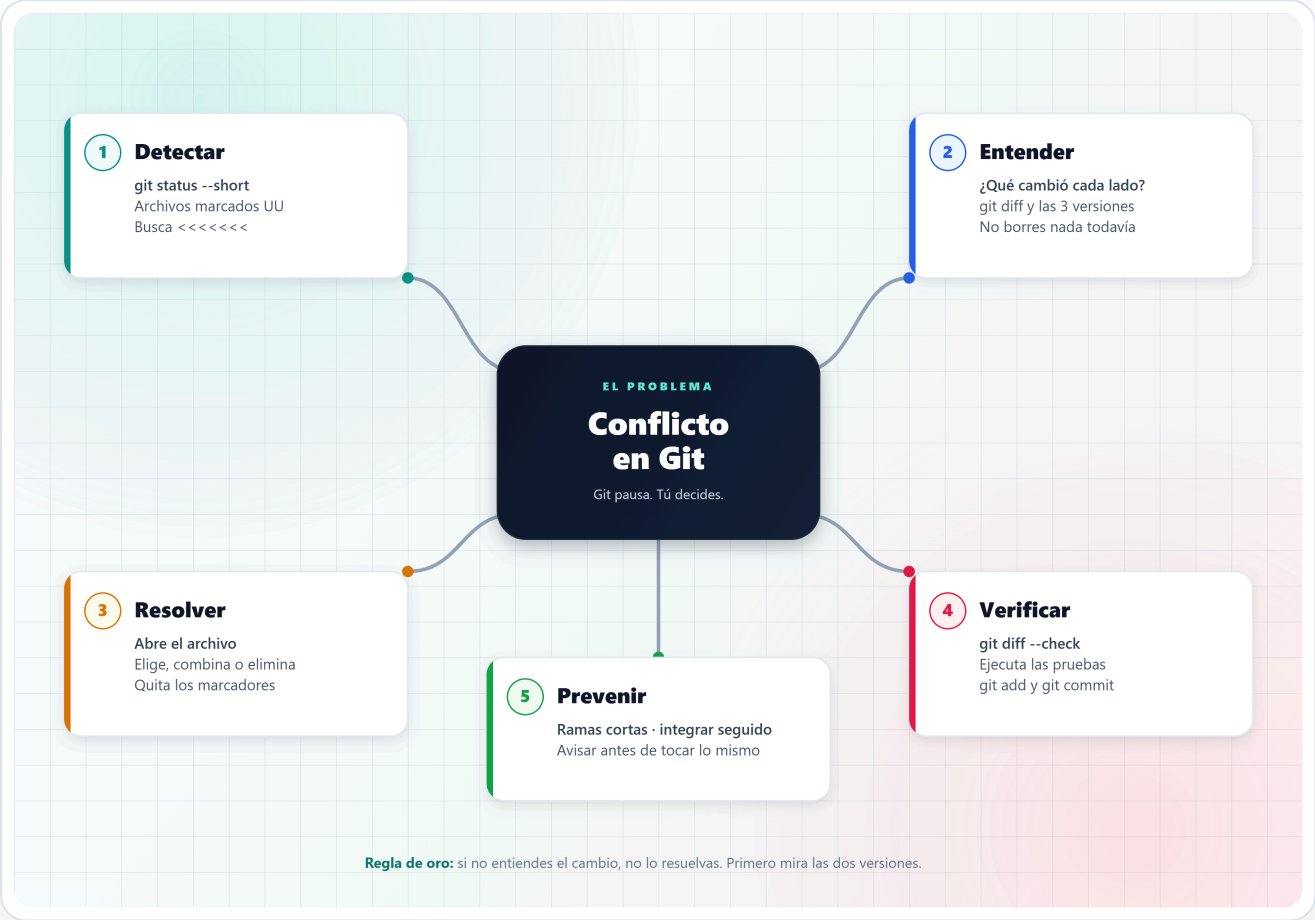
REGLA DE ORO

Si no entiendes el cambio, no lo resuelvas. Primero mira las dos versiones.

MAPA MENTAL

Cómo resolver un conflicto en cinco miradas

Las cuatro primeras ramas son el ciclo de resolución. La quinta es el hábito que evita repetirlo.



1 Detectar
Estado, archivos UU y marcas en el archivo.

2 Entender
Qué quería cada lado y por qué.

3 Resolver
Una decisión por cada bloque.

4 Verificar
Sin marcas, con pruebas y commit.

5 Prevenir
Ramas cortas, avisos e integración frecuente.

REGLA DE ORO

Si no entiendes el cambio, no lo resuelvas. Primero mira las dos versiones.

Mapa dibujado con **Canvas HTML5** en `mapa-mental/mapa-mental.html`. Se puede abrir en el navegador y exportar en alta resolución.

EL MÉTODO

Siempre el mismo orden

Si te pierdes a mitad de camino, vuelve al paso 1. El orden evita la mayoría de los errores.

1

Detectar

Mira el estado y localiza los archivos marcados.

`git status --short`

2

Entender

Compara las dos versiones antes de editar.

`git diff`

3

Resolver

Elige o combina; elimina todos los marcadores.

`editor + git add`

4

Verificar

Confirma que no quedan marcas y que todo pasa.

`git diff --check`

5 · Prevenir Ramas cortas, integrar seguido y avisar cuando dos personas tocan el mismo archivo.

LOS 10 EJERCICIOS DE ESTA GUÍA

#	Ejercicio	Operación	Resultado
01	**Conflicto simple en una línea**	merge	**5 PASS · 0 FAIL**
02	**Varios conflictos en un archivo**	merge	**6 PASS · 0 FAIL**
03	**Conflicto modificar / eliminar**	merge	**4 PASS · 0 FAIL**
04	**Conflicto añadir / añadir**	merge	**5 PASS · 0 FAIL**
05	**Conflicto con un archivo binario**	merge	**4 PASS · 0 FAIL**
06	**Conflicto durante un rebase**	rebase	**5 PASS · 0 FAIL**
07	**Conflicto en un cherry-pick**	cherry-pick	**5 PASS · 0 FAIL**
08	**Conflicto al recuperar un stash**	stash pop	**4 PASS · 0 FAIL**
09	**Inspección y herramienta de merge**	mergetool	**4 PASS · 0 FAIL**
10	**Estrategias automáticas**	estrategias	**15 PASS · 0 FAIL**

Cómo usar la guía. Lee la página del ejercicio, revisa la secuencia de capturas y repite los pasos en un repositorio de prueba. Cada secuencia muestra los comandos tal como se ejecutaron en el equipo.

LOS ESTADOS QUE APARECEN EN GIT STATUS

UU

Los dos modificaron el archivo.

AA

Los dos añadieron el mismo archivo.

UD

Uno modificó y el otro eliminó.

DU

Uno eliminó y el otro modificó.

EJERCICIO 01 / 10 · BÁSICA

MERGE

Conflicto simple en una línea

QUÉ ESTÁ PASANDO

Las dos ramas cambiaron la misma línea de `app.txt`. Git no sabe cuál elegir y deja las dos versiones dentro del archivo, entre las marcas `<<<<<<<`, `=======` y `>>>>>>>`.

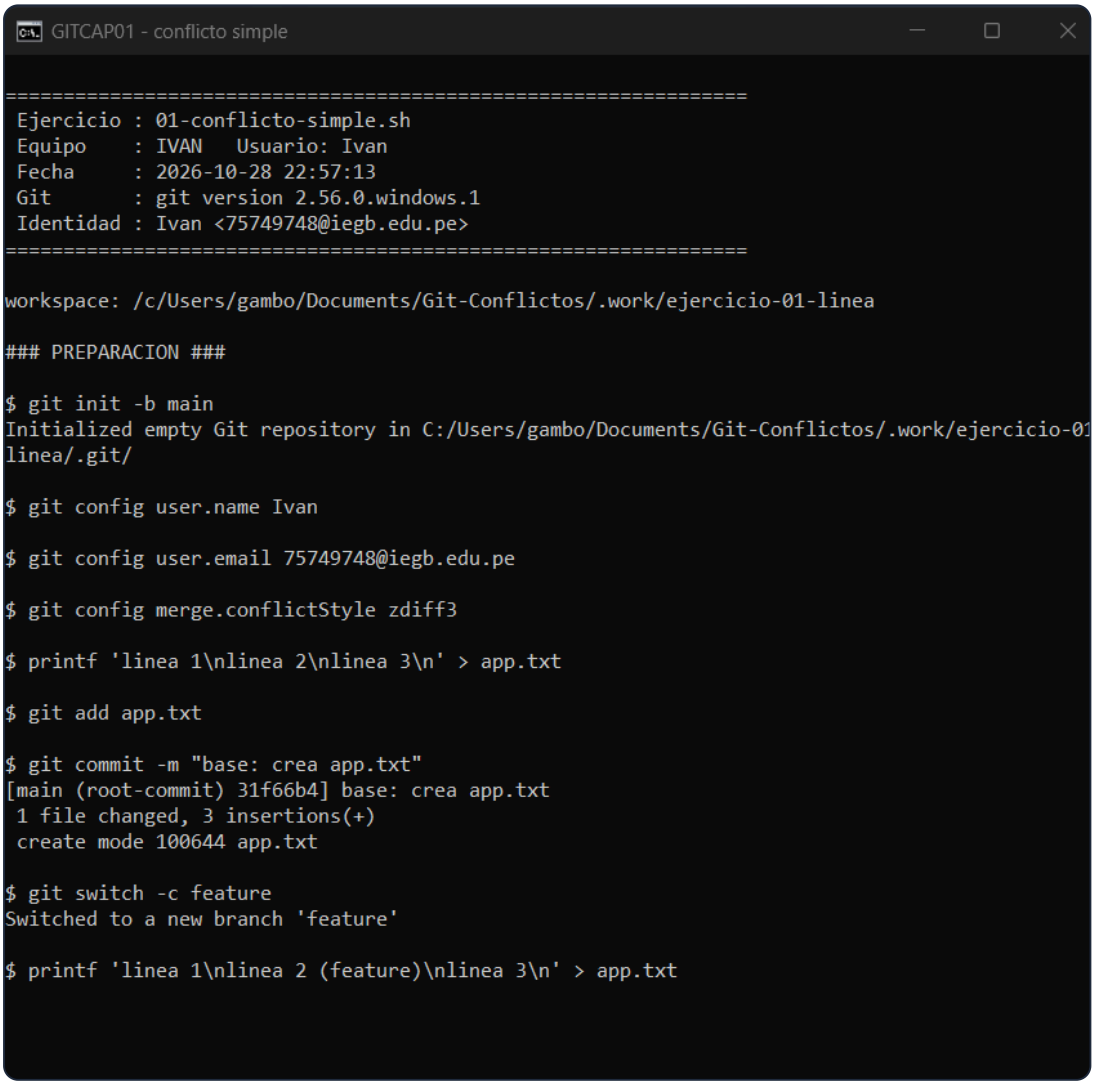
CÓMO SE RESUELVE

1. El archivo aparece como **UU**: lo modificaron los dos lados.
   `git status --short`

2. Abre el archivo y conserva las dos líneas: **linea 2 (main)** y **linea 2 (feature)**.
   `app.txt`

3. Borra las marcas, guarda y confirma el merge.
   `git add + git commit`

5 PASS | **0** FAIL — Árbol limpio. app.txt conserva los dos cambios y no quedan marcadores.



Captura real 1 de 4 · consola del equipo — 5 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 4 completan la secuencia del ejercicio 01.

GITCAP01 - conflicto simple

```
Switched to a new branch 'feature'

$ printf 'linea 1\nlinea 2 (feature)\nlinea 3\n' > app.txt

$ git commit -am "feature: cambia la linea 2"
[feature 11fd76a] feature: cambia la linea 2
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git switch main
Switched to branch 'main'

$ printf 'linea 1\nlinea 2 (main)\nlinea 3\n' > app.txt

$ git commit -am "main: cambia la linea 2"
[main f3de28c] main: cambia la linea 2
 1 file changed, 1 insertion(+), 1 deletion(-)

### CONFLICTO ###

$ git merge feature
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
UU app.txt

$ git diff
diff --cc app.txt
index 528d47a,53fc77e..0000000
--- a/app.txt
+++ b/app.txt
@@@ -1,3 -1,3 +1,9 @@@
  linea 1
++<<<<<<<< HEAD
 +linea 2 (main)
++|||||||| 31f66b4
++linea 2
```

Captura 2 de 4 comando y salida reales

GITCAP01 - conflicto simple

```
 +linea 2 (main)
++|||||||| 31f66b4
++linea 2
++========
+ linea 2 (feature)
++>>>>>>>> feature
  linea 3

$ git show :1:app.txt
linea 1
linea 2
linea 3

$ git show :2:app.txt
linea 1
linea 2 (main)
linea 3

$ git show :3:app.txt
linea 1
linea 2 (feature)
linea 3

$ cat app.txt
linea 1
<<<<<<< HEAD
linea 2 (main)
||||||| 31f66b4
linea 2
=======
linea 2 (feature)
>>>>>>> feature
linea 3

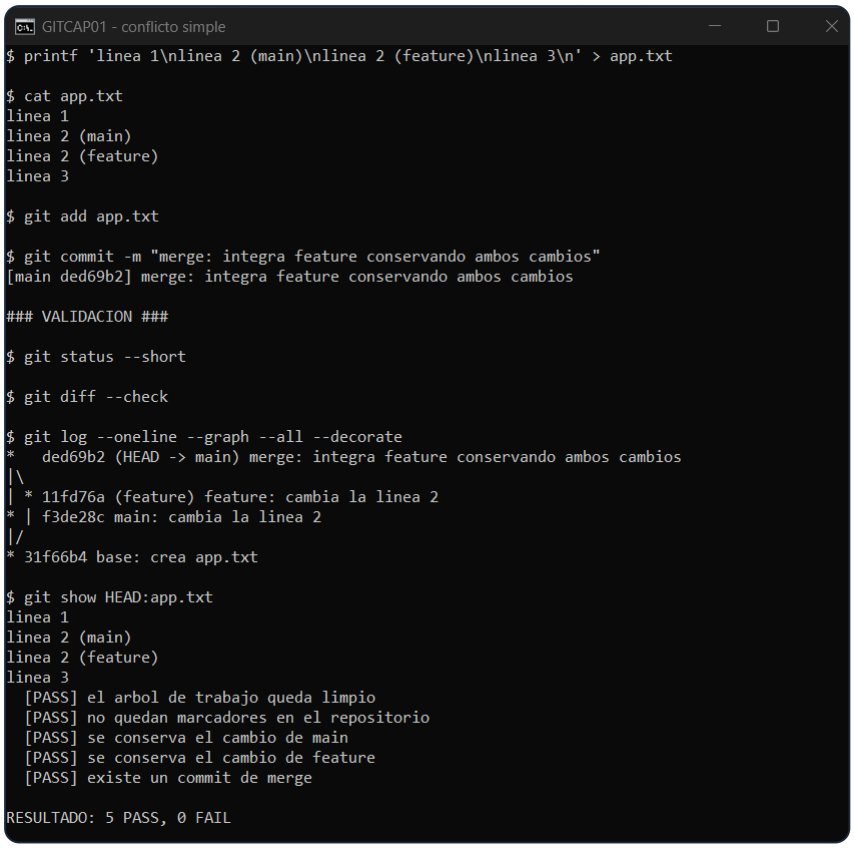
### RESOLUCION ###

$ printf 'linea 1\nlinea 2 (main)\nlinea 2 (feature)\nlinea 3\n' > app.txt

$ cat app.txt
```

Captura 3 de 4 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 de 4 completan la secuencia del ejercicio 01.



Captura 4 de 4 comando y salida reales

EJERCICIO 02 / 10 · INTERMEDIA

MERGE

Varios conflictos en un archivo

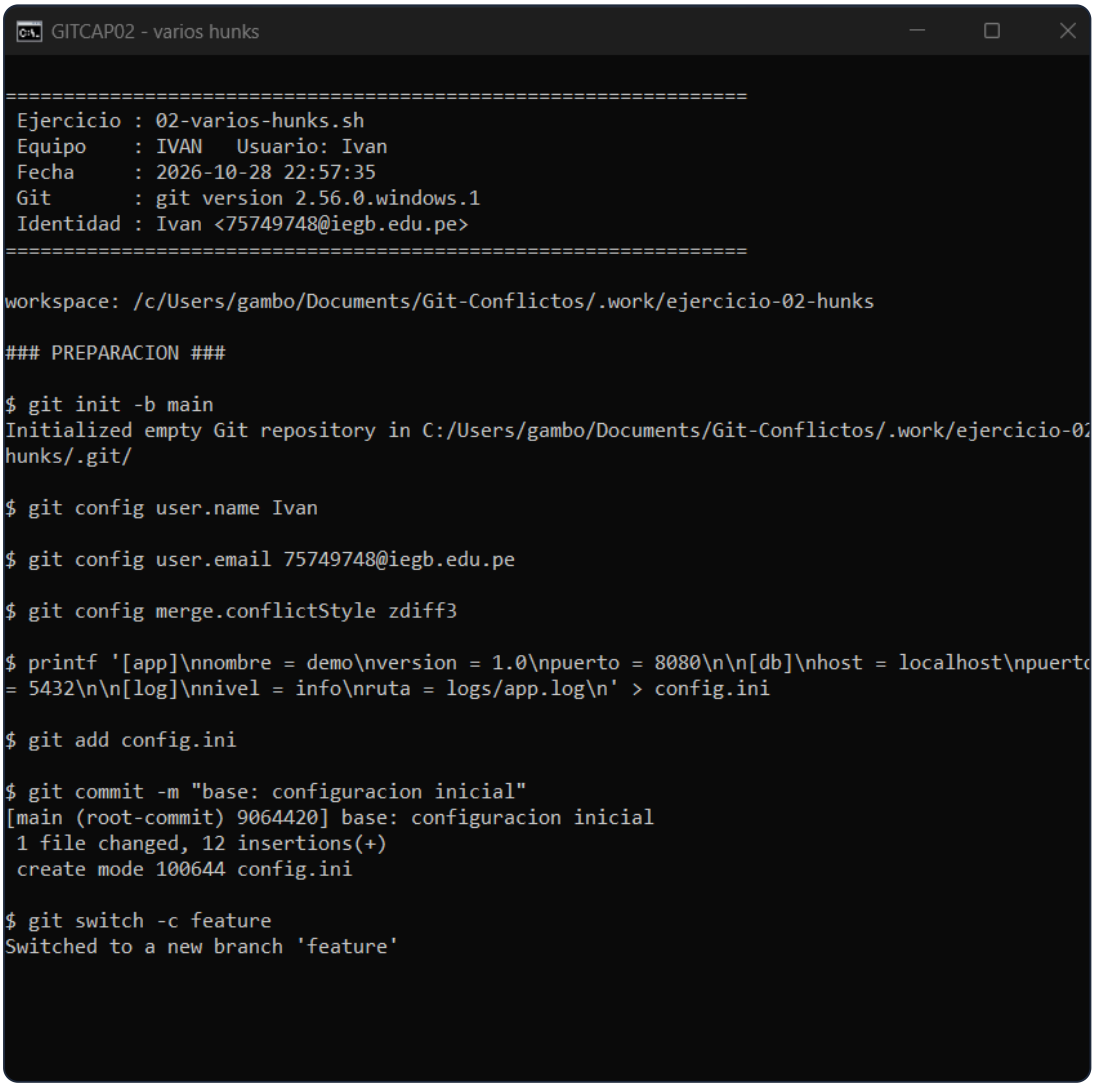
QUÉ ESTÁ PASANDO

El mismo `config.ini` tiene dos zonas en conflicto: el nombre de la aplicación y el nivel de log. Cada zona se decide por separado; no conviene aplicar una sola respuesta a todo el archivo.

CÓMO SE RESUELVE

1. Localiza los dos bloques de conflicto y lee qué cambió cada lado.
   `git diff`
2. Combina el nombre como **demo (main+feature)** y elige **debug** para el log.
   `config.ini`
3. Quita las marcas, guarda y confirma.
   `git add + git commit`

6 PASS | **0** FAIL — Árbol limpio. Las dos zonas quedaron resueltas con decisiones independientes.



Captura real 1 de 5 · consola del equipo — 6 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 5 completan la secuencia del ejercicio 02.

GITCAP02 - varios hunks

```
$ git switch -c feature
Switched to a new branch 'feature'

$ printf '[app]\nnombre = demo-feature\nversion = 1.0\npuerto = 8080\n\n[db]\nhost = localhost
npuerto = 5432\n\n[log]\nnivel = debug\nruta = logs/app.log\n' > config.ini

$ git commit -am "feature: nombre y nivel debug"
[feature ac221ea] feature: nombre y nivel debug
 1 file changed, 2 insertions(+), 2 deletions(-)

$ git switch main
Switched to branch 'main'

$ printf '[app]\nnombre = demo-main\nversion = 1.0\npuerto = 8080\n\n[db]\nhost = localhost\np
erto = 5432\n\n[log]\nnivel = warn\nruta = logs/app.log\n' > config.ini

$ git commit -am "main: nombre y nivel warn"
[main b4eb857] main: nombre y nivel warn
 1 file changed, 2 insertions(+), 2 deletions(-)

### CONFLICTO ###

$ git merge feature
Auto-merging config.ini
CONFLICT (content): Merge conflict in config.ini
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
UU config.ini

$ git diff
diff --cc config.ini
index e1eabd0,5443e3b..0000000
--- a/config.ini
+++ b/config.ini
@@@ -1,5 -1,5 +1,11 @@@
  [app]
++<<<<<<< HEAD
```

Captura 2 de 5 comando y salida reales

GITCAP02 - varios hunks

```
@@@ -1,5 -1,5 +1,11 @@@
  [app]
++<<<<<<< HEAD
 +nombre = demo-main
++||||||| 9064420
++nombre = demo
++=======
+ nombre = demo-feature
++>>>>>>> feature
  version = 1.0
  puerto = 8080

@@@ -8,5 -8,5 +14,11 @@@ host = localhos
  puerto = 5432

  [log]
++<<<<<<< HEAD
 +nivel = warn
++||||||| 9064420
++nivel = info
++=======
+ nivel = debug
++>>>>>>> feature
  ruta = logs/app.log
  [PASS] hay dos regiones de conflicto

### RESOLUCION ###

$ printf '[app]\nnombre = demo (main+feature)\nversion = 1.0\npuerto = 8080\n\n[db]\nhost = lo
alhost\npuerto = 5432\n\n[log]\nnivel = debug\nruta = logs/app.log\n' > config.ini

$ cat config.ini
[app]
nombre = demo (main+feature)
version = 1.0
puerto = 8080

[db]
host = localhost
```

Captura 3 de 5 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 y 5 de 5 completan la secuencia del ejercicio 02.

GITCAP02 - varios hunks

```
[db]
host = localhost
puerto = 5432

[log]
nivel = debug
ruta = logs/app.log

$ git add config.ini

$ git commit -m "merge: combina nombre y elige nivel debug"
[main 34f9a85] merge: combina nombre y elige nivel debug

### VALIDACION ###

$ git status --short

$ git diff --check

$ git show HEAD:config.ini
[app]
nombre = demo (main+feature)
version = 1.0
puerto = 8080

[db]
host = localhost
puerto = 5432

[log]
nivel = debug
ruta = logs/app.log
  [PASS] el arbol de trabajo queda limpio
  [PASS] no quedan marcadores en el repositorio
  [PASS] el nombre combina ambos lados
  [PASS] se eligio el nivel debug
  [PASS] no se conserva el nivel warn
```

Captura 4 de 5 comando y salida reales

GITCAP02 - varios hunks

```
[db]
host = localhost
puerto = 5432

[log]
nivel = debug
ruta = logs/app.log

$ git add config.ini

$ git commit -m "merge: combina nombre y elige nivel debug"
[main 34f9a85] merge: combina nombre y elige nivel debug

### VALIDACION ###

$ git status --short

$ git diff --check

$ git show HEAD:config.ini
[app]
nombre = demo (main+feature)
version = 1.0
puerto = 8080

[db]
host = localhost
puerto = 5432

[log]
nivel = debug
ruta = logs/app.log
  [PASS] el arbol de trabajo queda limpio
  [PASS] no quedan marcadores en el repositorio
  [PASS] el nombre combina ambos lados
  [PASS] se eligio el nivel debug
  [PASS] no se conserva el nivel warn

RESULTADO: 6 PASS, 0 FAIL
```

Captura 5 de 5 comando y salida reales

EJERCICIO 03 / 10 · INTERMEDIA

MERGE

Conflicto modificar / eliminar

QUÉ ESTÁ PASANDO

Una rama editó `notas.txt` y la otra lo eliminó. Git pregunta, en la práctica, si el archivo debe seguir existiendo. Es el estado **UD** y la decisión es de producto, no de sintaxis.

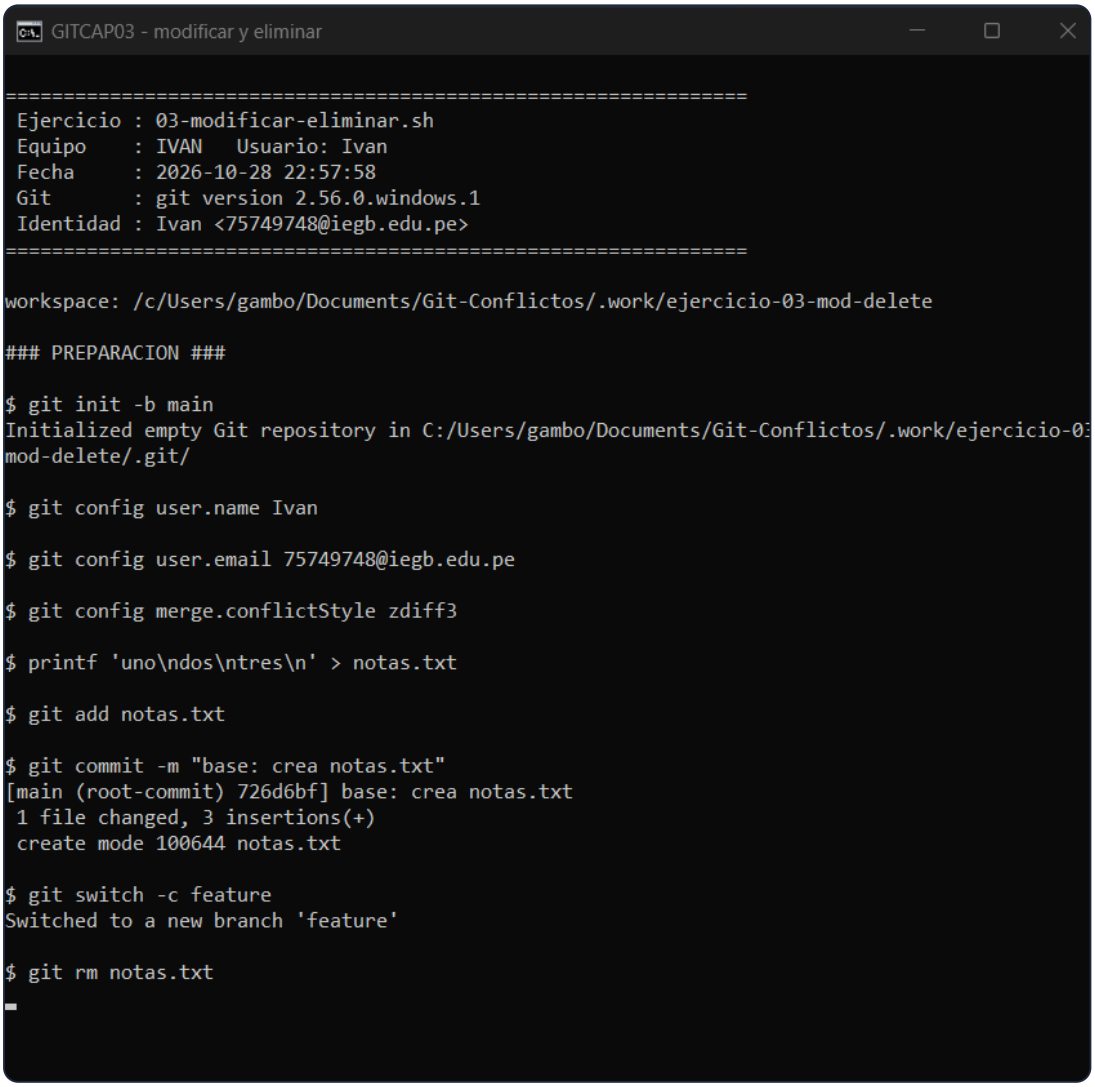
CÓMO SE RESUELVE

1. Comprueba el estado y lee el contenido que quiso conservar una de las ramas.
   `git status + cat`

2. Elige: conservar el archivo con **add** o aceptar la eliminación con **rm**.
   `git add · git rm`

3. Confirma el merge con la decisión tomada.
   `git commit`

4 PASS | **0** FAIL — Árbol limpio. Se documentó la solución de eliminar notas.txt.



Captura real 1 de 5 · consola del equipo — 4 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 5 completan la secuencia del ejercicio 03.

GITCAP03 - modificar y eliminar

```
Switched to a new branch 'feature'

$ git rm notas.txt
rm 'notas.txt'

$ git commit -m "feature: elimina notas.txt"
[feature e8853fb] feature: elimina notas.txt
 1 file changed, 3 deletions(-)
 delete mode 100644 notas.txt

$ git switch main
Switched to branch 'main'

$ printf 'uno\ndos (main)\ntres\n' > notas.txt

$ git commit -am "main: edita notas.txt"
[main 85550ee] main: edita notas.txt
 1 file changed, 1 insertion(+), 1 deletion(-)

### CONFLICTO ###

$ git merge feature
CONFLICT (modify/delete): notas.txt deleted in feature and modified in HEAD.  Version HEAD of
otas.txt left in tree.
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
UD notas.txt

$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add/rm <file>..." as appropriate to mark resolution)
        deleted by them: notas.txt
```

Captura 2 de 5 comando y salida reales

GITCAP03 - modificar y eliminar

```
Unmerged paths:
  (use "git add/rm <file>..." as appropriate to mark resolution)
        deleted by them: notas.txt

no changes added to commit (use "git add" and/or "git commit -a")

$ cat notas.txt
uno
dos (main)
tres

$ git log --merge --oneline -- notas.txt
85550ee main: edita notas.txt
e8853fb feature: elimina notas.txt

### RESOLUCION A - CONSERVAR ###

$ git add notas.txt

$ git commit -m "merge: conserva notas.txt con la edicion de main"
[main d9cab2c] merge: conserva notas.txt con la edicion de main

$ cat notas.txt
uno
dos (main)
tres

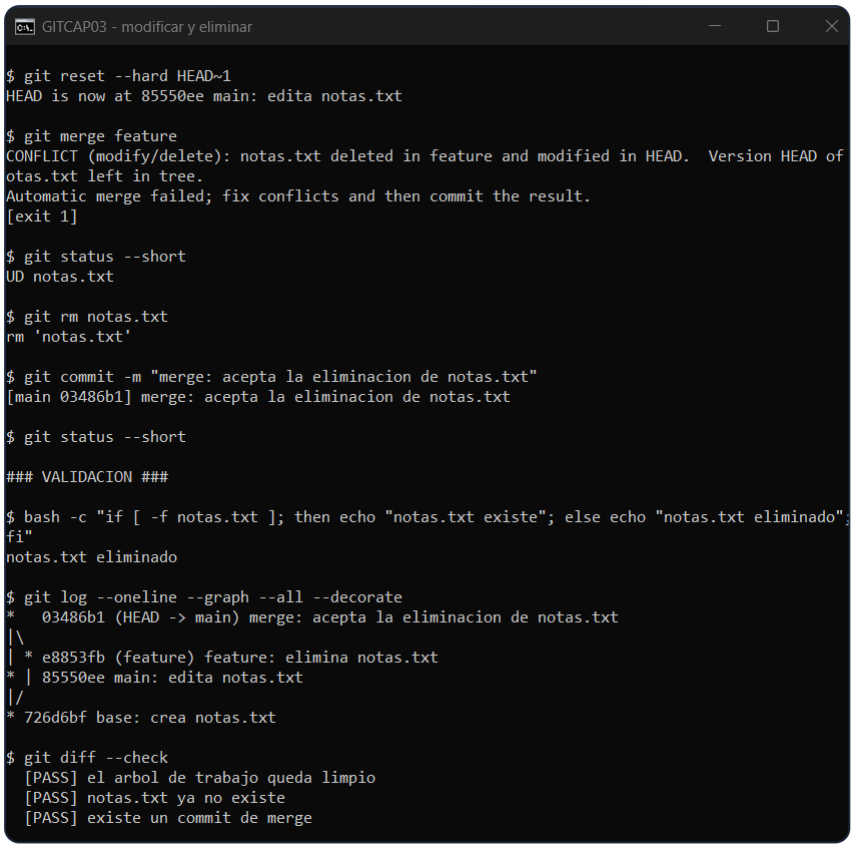
$ git log --oneline --graph --all --decorate
*   d9cab2c (HEAD -> main) merge: conserva notas.txt con la edicion de main
|\
| * e8853fb (feature) feature: elimina notas.txt
* | 85550ee main: edita notas.txt
|/
* 726d6bf base: crea notas.txt

### RESOLUCION B - ELIMINAR ###

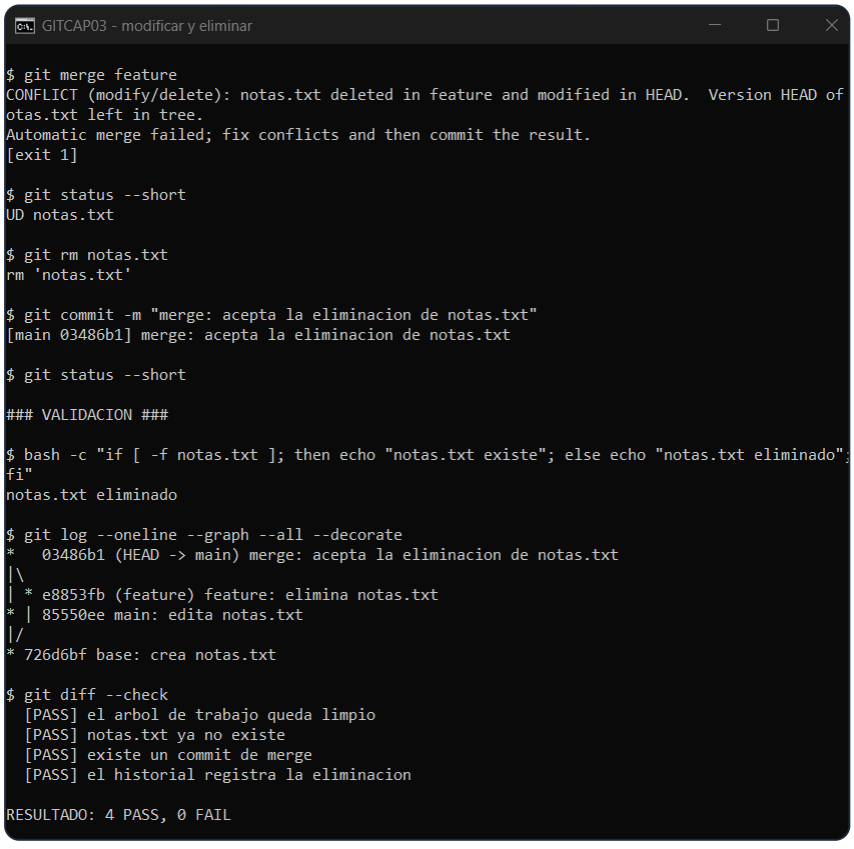
$ git reset --hard HEAD~1
HEAD is now at 85550ee main: edita notas.txt
```

Captura 3 de 5 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 y 5 de 5 completan la secuencia del ejercicio 03.



Captura 4 de 5 comando y salida reales



Captura 5 de 5 comando y salida reales

EJERCICIO 04 / 10 · INTERMEDIA

MERGE

Conflicto añadir / añadir

QUÉ ESTÁ PASANDO

Dos ramas crearon el mismo archivo nuevo, `CHANGELOG.md`, con contenidos distintos. Es el estado **AA**: no existe una versión base del archivo que sirva de referencia.

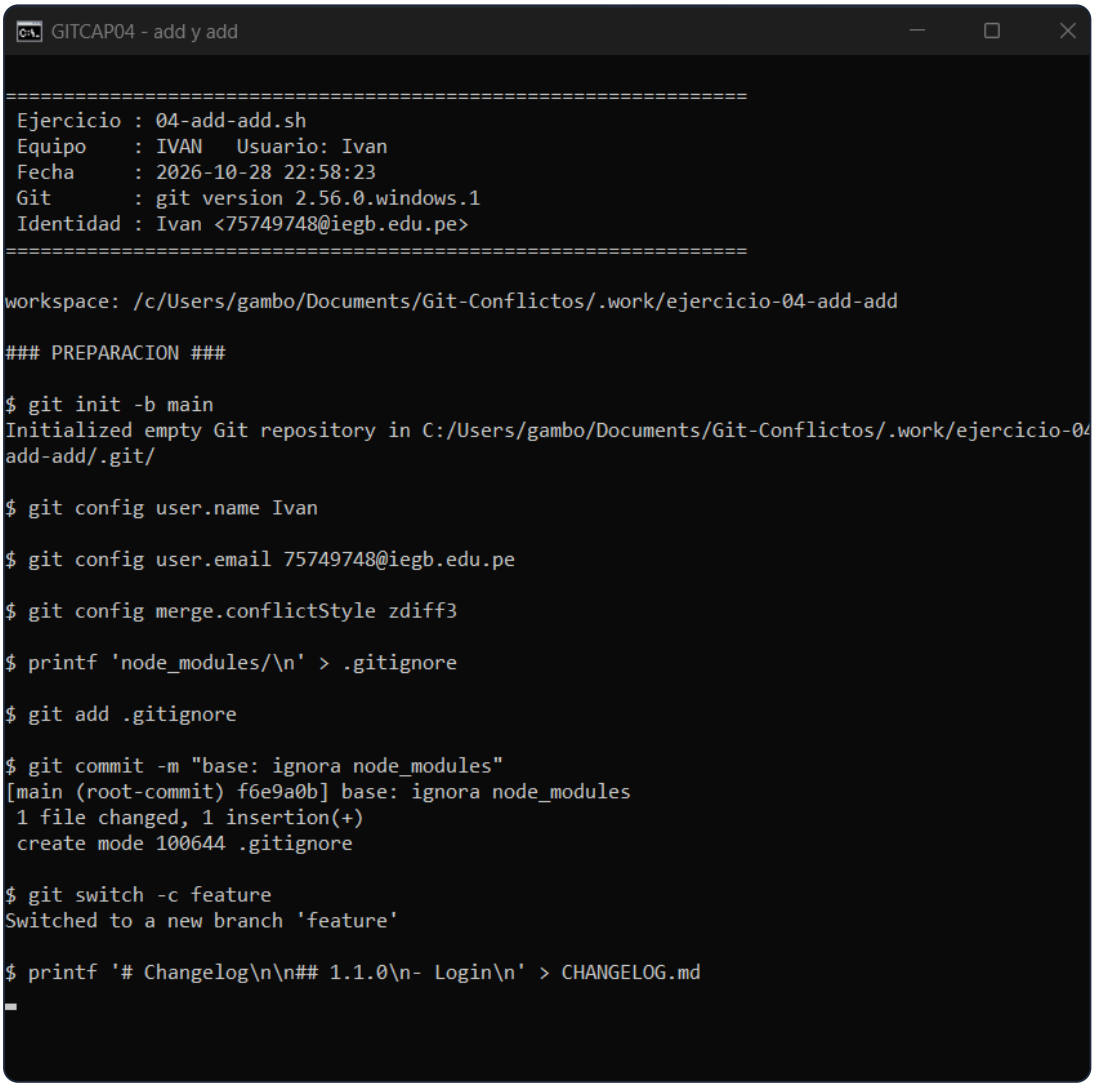
CÓMO SE RESUELVE

1. Git pegó los dos documentos completos. Léelos antes de tocar nada.
   `cat CHANGELOG.md`

2. Ordena las entradas **1.1.0** y **1.0.1** en un único changelog.
   `CHANGELOG.md`

3. Guarda el archivo unificado y confirma.
   `git add + git commit`

5 PASS | **0** FAIL — Árbol limpio. Un solo CHANGELOG con las dos entradas.



Captura real 1 de 4 · consola del equipo — 5 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 4 completan la secuencia del ejercicio 04.

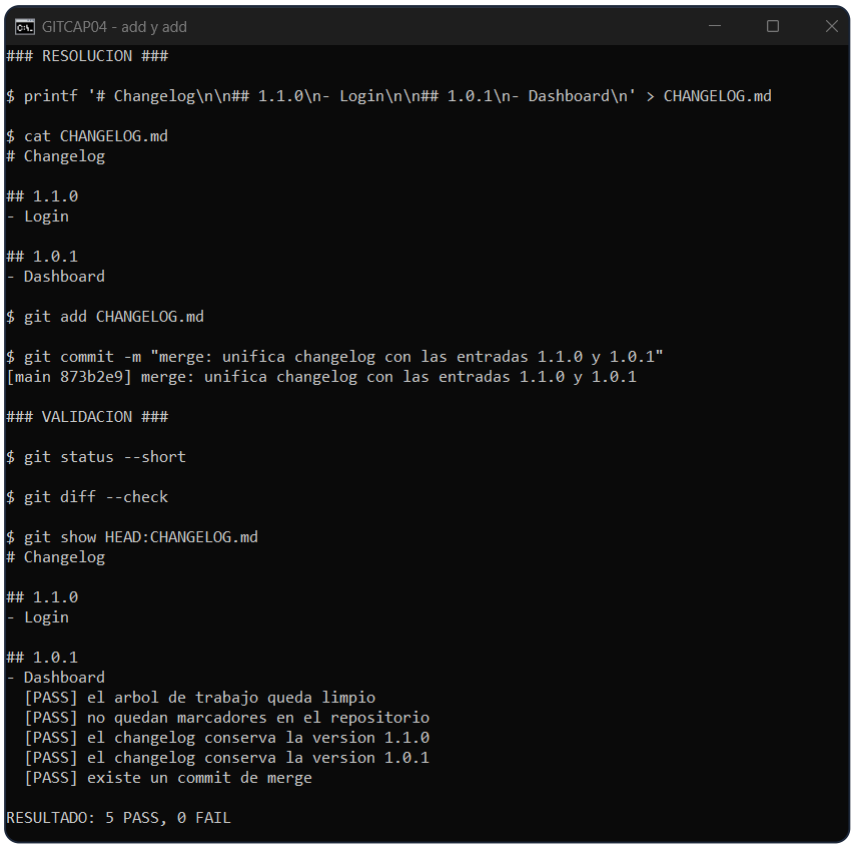


Captura 2 de 4 comando y salida reales



Captura 3 de 4 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 de 4 completan la secuencia del ejercicio 04.



Captura 4 de 4 comando y salida reales

EJERCICIO 05 / 10 · INTERMEDIA

MERGE

Conflicto con un archivo binario

QUÉ ESTÁ PASANDO

`logo.bin` es binario. Git no puede comparar su contenido ni mostrar marcas: solo puede quedarse con una versión completa. Para conservar las dos, habría que renombrar una antes de integrar.

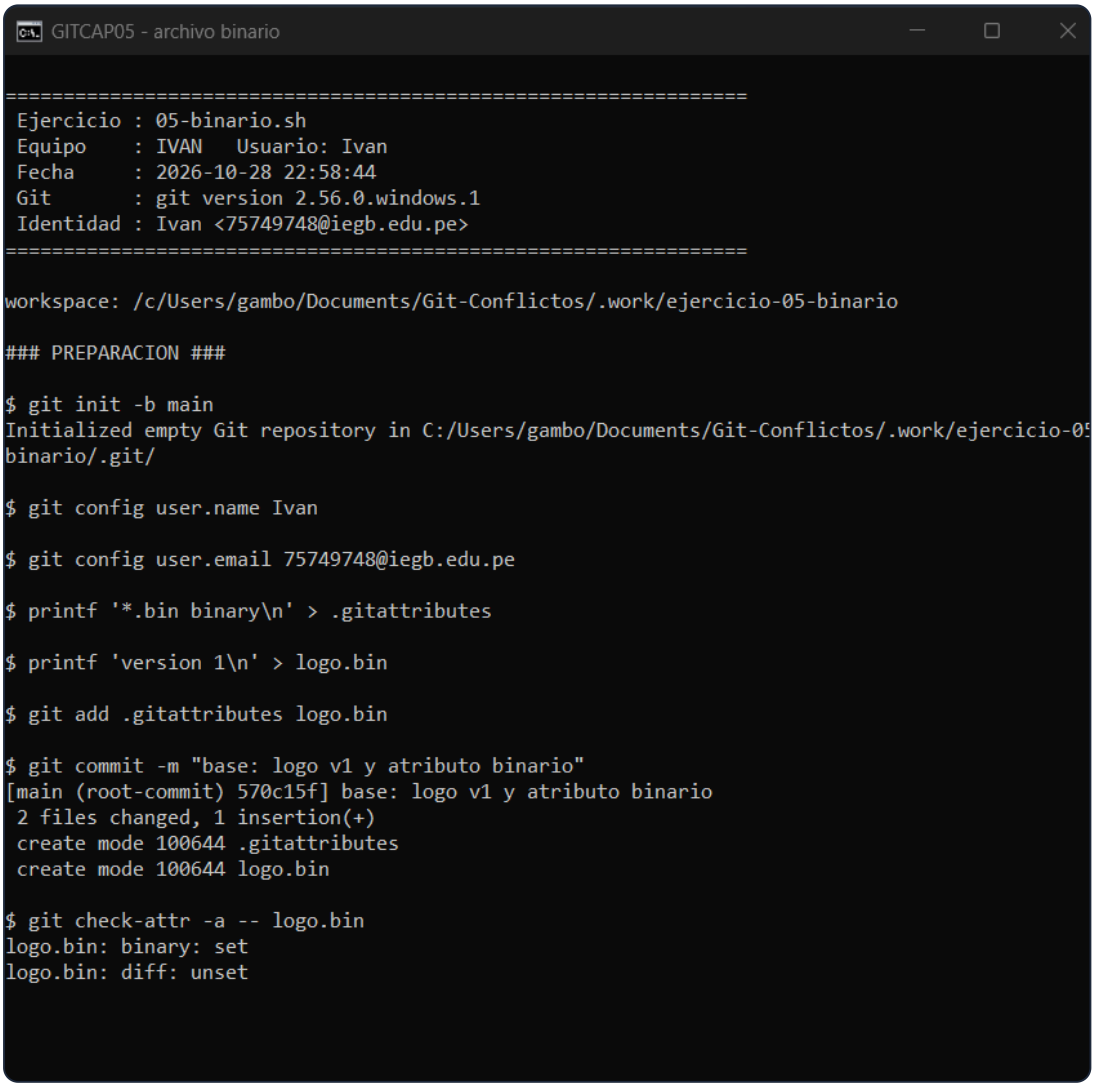
CÓMO SE RESUELVE

1. Aparece el aviso **Cannot merge binary files**. No hay texto que editar.
   `git diff`

2. Elige una versión completa del archivo (aquí, la de **feature**).
   `git checkout --theirs`

3. Marca la versión elegida y confirma el merge.
   `git add + git commit`

4 PASS | **0** FAIL — Árbol limpio. logo.bin quedó con la versión de feature.



Captura real 1 de 4 · consola del equipo — 4 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 4 completan la secuencia del ejercicio 05.

GITCAP05 - archivo binario

```
$ git check-attr -a -- logo.bin
logo.bin: binary: set
logo.bin: diff: unset
logo.bin: merge: unset
logo.bin: text: unset

$ git switch -c feature
Switched to a new branch 'feature'

$ printf 'version FEATURE\n' > logo.bin

$ git commit -am "feature: version FEATURE del logo"
[feature 994f6f2] feature: version FEATURE del logo
 1 file changed, 0 insertions(+), 0 deletions(-)

$ git switch main
Switched to branch 'main'

$ printf 'version MAIN\n' > logo.bin

$ git commit -am "main: version MAIN del logo"
[main 975b048] main: version MAIN del logo
 1 file changed, 0 insertions(+), 0 deletions(-)

### CONFLICTO ###

$ git merge feature
warning: Cannot merge binary files: logo.bin (HEAD vs. feature)
Auto-merging logo.bin
CONFLICT (content): Merge conflict in logo.bin
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
UU logo.bin

$ git diff
diff --cc logo.bin
index 4ed3b23,2ce1430..0000000
```

Captura 2 de 4 comando y salida reales

GITCAP05 - archivo binario

```
$ git diff
diff --cc logo.bin
index 4ed3b23,2ce1430..0000000
Binary files differ

$ cat logo.bin
version MAIN

### RESOLUCION ###

$ git checkout --theirs -- logo.bin

$ cat logo.bin
version FEATURE

$ git add logo.bin

$ git commit -m "merge: usa la version FEATURE del logo binario"
[main 036b088] merge: usa la version FEATURE del logo binario

### VALIDACION ###

$ git status --short

$ git diff --check

$ cat logo.bin
version FEATURE

$ git log --oneline --graph --all --decorate
*   036b088 (HEAD -> main) merge: usa la version FEATURE del logo binario
|\
| * 994f6f2 (feature) feature: version FEATURE del logo
* | 975b048 main: version MAIN del logo
|/
* 570c15f base: logo v1 y atributo binario
  [PASS] el arbol de trabajo queda limpio
  [PASS] se eligio la version FEATURE
  [PASS] no se conserva la version MAIN
```

Captura 3 de 4 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 de 4 completan la secuencia del ejercicio 05.

GITCAP05 - archivo binario

```
Binary files differ

$ cat logo.bin
version MAIN

### RESOLUCION ###

$ git checkout --theirs -- logo.bin

$ cat logo.bin
version FEATURE

$ git add logo.bin

$ git commit -m "merge: usa la version FEATURE del logo binario"
[main 036b088] merge: usa la version FEATURE del logo binario

### VALIDACION ###

$ git status --short

$ git diff --check

$ cat logo.bin
version FEATURE

$ git log --oneline --graph --all --decorate
*   036b088 (HEAD -> main) merge: usa la version FEATURE del logo binario
|\
| * 994f6f2 (feature) feature: version FEATURE del logo
* | 975b048 main: version MAIN del logo
|/
* 570c15f base: logo v1 y atributo binario
  [PASS] el arbol de trabajo queda limpio
  [PASS] se eligio la version FEATURE
  [PASS] no se conserva la version MAIN
  [PASS] existe un commit de merge

RESULTADO: 4 PASS, 0 FAIL
```

Captura 4 de 4 comando y salida reales

EJERCICIO 06 / 10 · AVANZADA

REBASE

Conflicto durante un rebase

QUÉ ESTÁ PASANDO

En un rebase, Git reaplica tus commits sobre otra base. Si chocan, **ours** es la rama base y **theirs** es tu commit: justo al revés que en un merge.

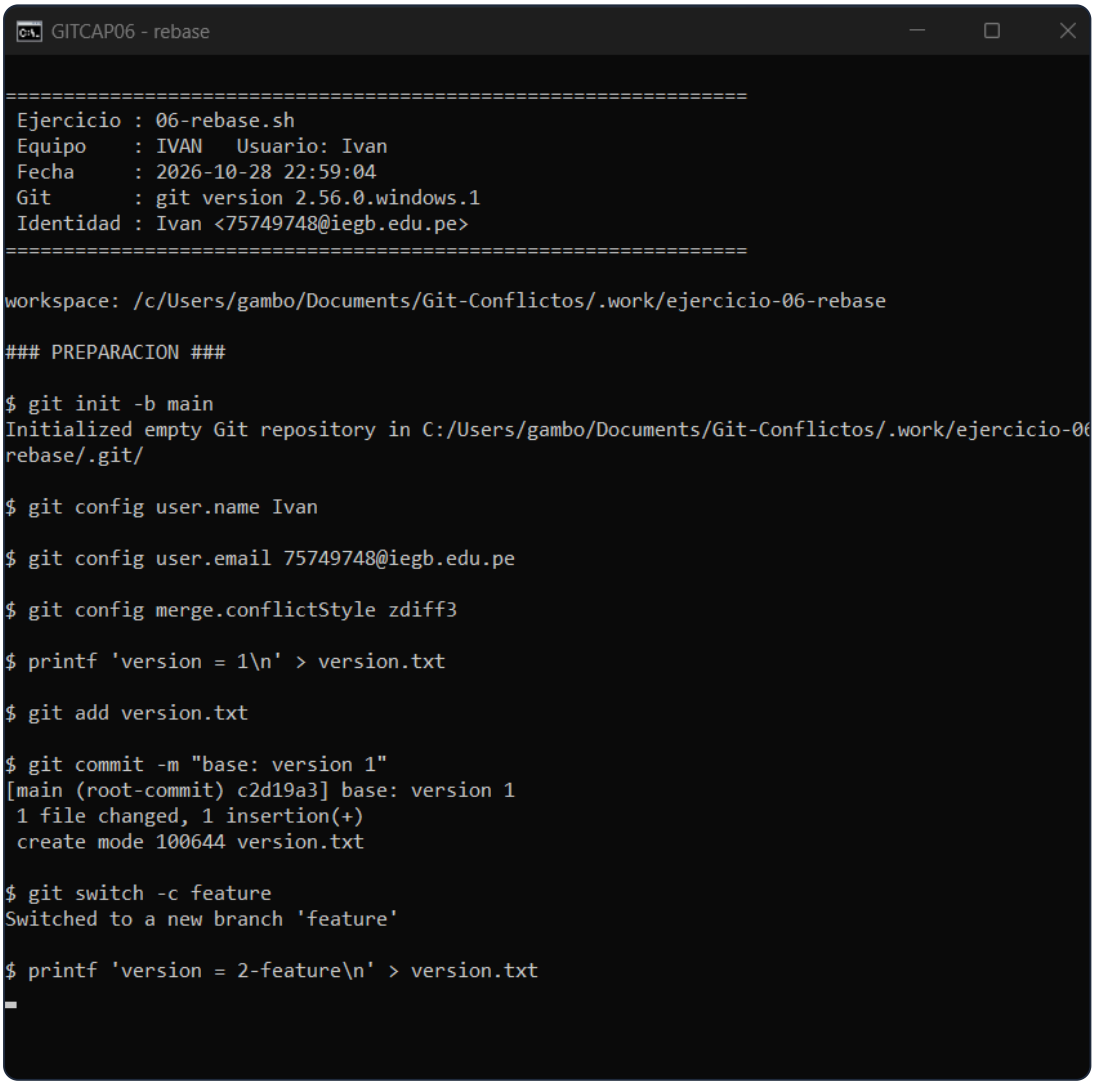
CÓMO SE RESUELVE

1. Comprueba qué archivo detuvo el rebase.
   `git status`

2. Compara la base (main) con tu commit (feature) y resuelve el archivo.
   `git show :2: :3:`

3. Marca la resolución y continúa el rebase.
   `git add + git rebase --continue`

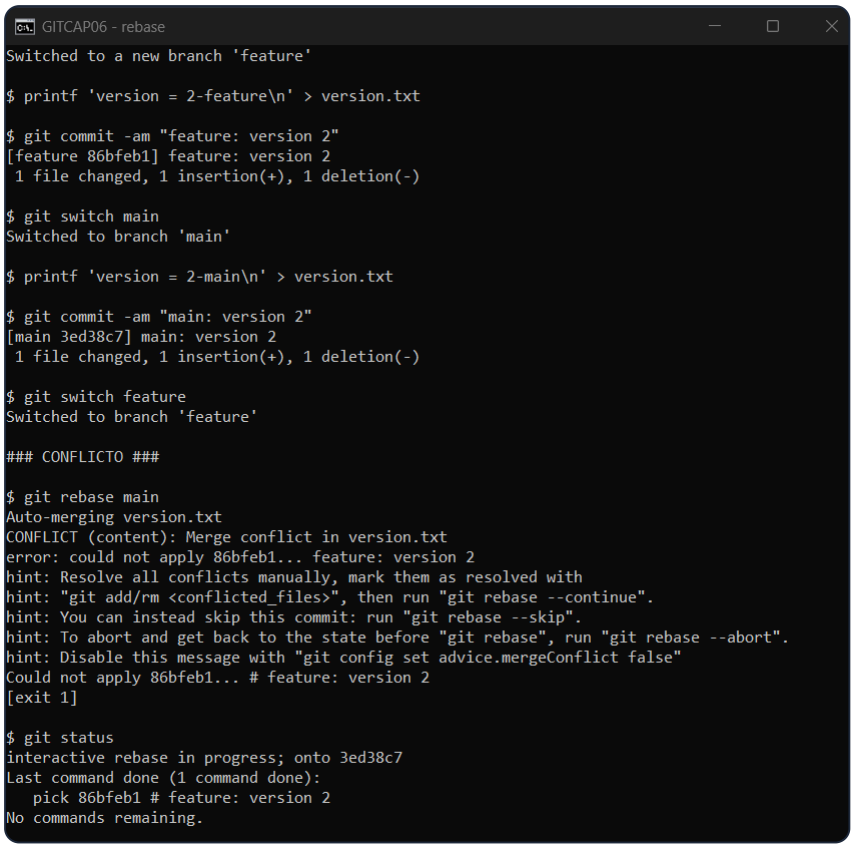
5 PASS | **0** FAIL — Historia lineal. No hay merge commit y los cambios se conservan.



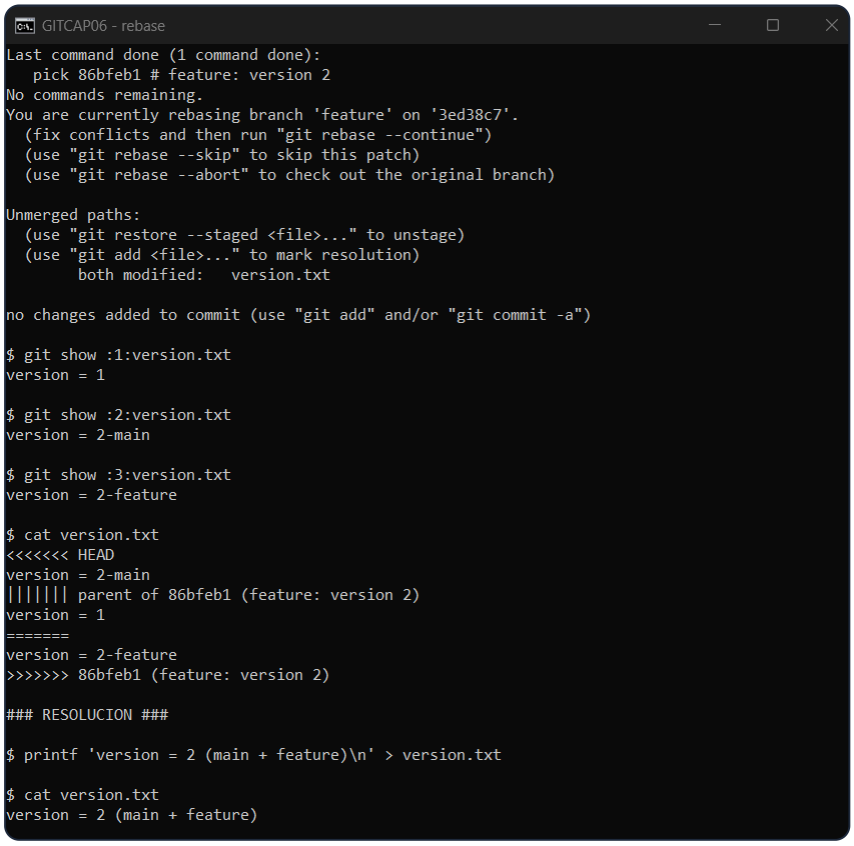
Captura real 1 de 4 · consola del equipo

5 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 4 completan la secuencia del ejercicio 06.

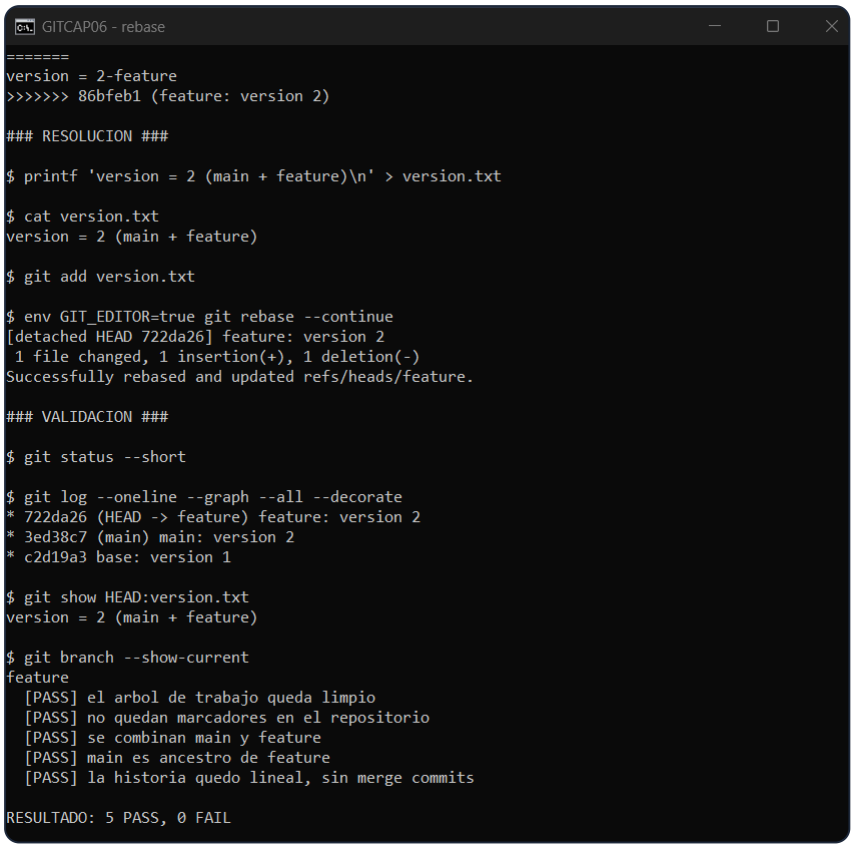


Captura 2 de 4 comando y salida reales



Captura 3 de 4 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 de 4 completan la secuencia del ejercicio 06.



Captura 4 de 4 comando y salida reales

EJERCICIO 07 / 10 · AVANZADA

CHERRY-PICK

Conflicto en un cherry-pick

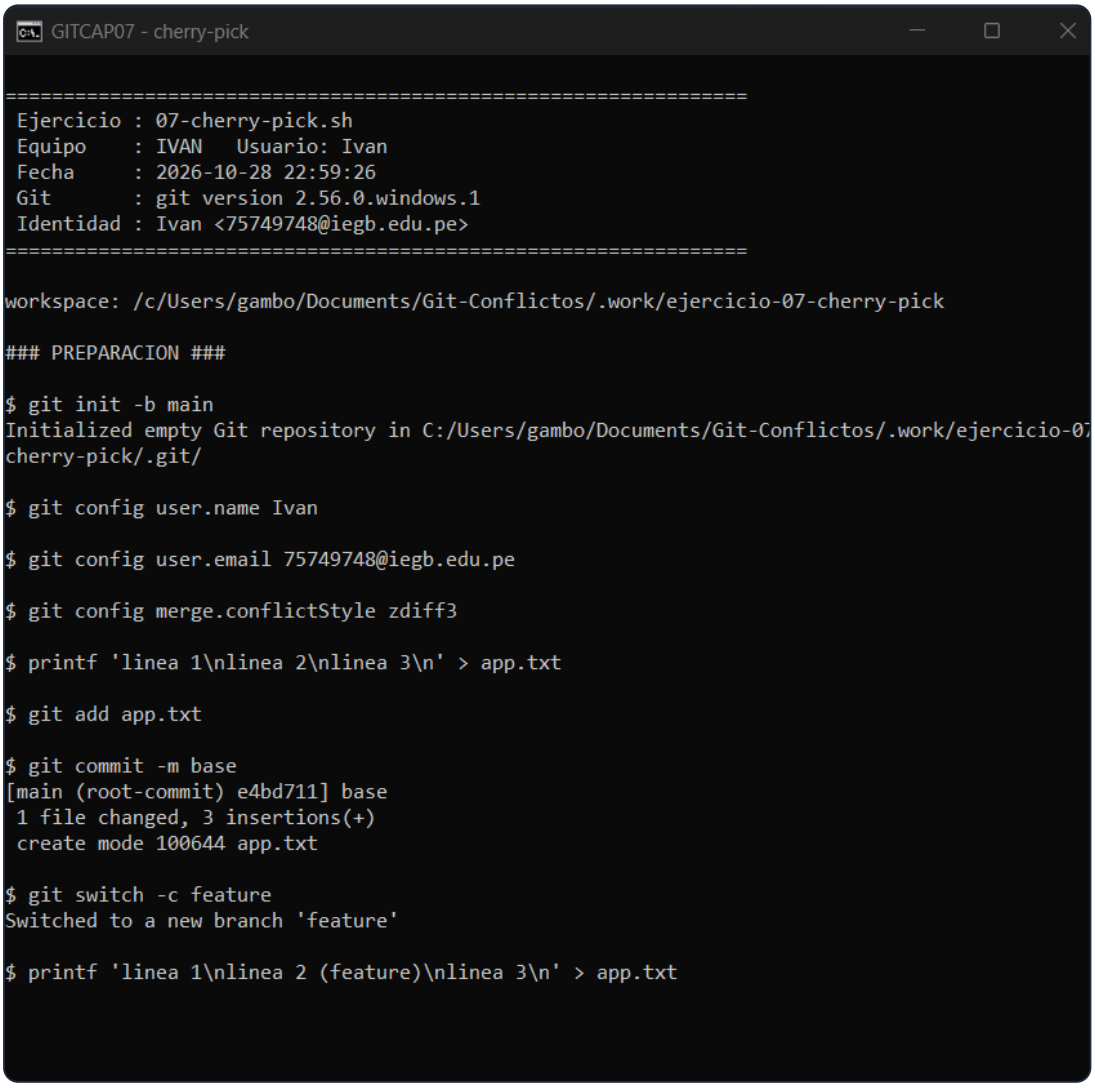
QUÉ ESTÁ PASANDO

Cherry-pick copia un commit concreto de `main` sobre `feature`. El parche ya no encaja en la nueva base y Git pausa para que lo ajustes.

CÓMO SE RESUELVE

1. Identifica el commit de main que quieres aplicar.
   `git log --oneline`
2. Aplícalo sobre feature y observa el conflicto.
   `git cherry-pick`
3. Resuelve app.txt, márcalo y continúa el cherry-pick.
   `git add + --continue`

5 PASS | **0** FAIL — Árbol limpio. El cambio se aplicó una sola vez, sin duplicarlo.



Captura real 1 de 5 · consola del equipo

5 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 5 completan la secuencia del ejercicio 07.

GITCAP07 - cherry-pick

```
$ printf 'linea 1\nlinea 2 (feature)\nlinea 3\n' > app.txt

$ git commit -am "feature: ajusta la linea 2"
[feature edeebec] feature: ajusta la linea 2
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git switch main
Switched to branch 'main'

$ printf 'linea 1\nlinea 2 (main)\nlinea 3\n' > app.txt

$ git commit -am "main: ajusta la linea 2"
[main fbd46ff] main: ajusta la linea 2
 1 file changed, 1 insertion(+), 1 deletion(-)

commit a aplicar: fbd46ffff833cd697e9a036d71bace4af57c84fa

$ git switch feature
Switched to branch 'feature'

### CONFLICTO ###

$ git cherry-pick fbd46ffff833cd697e9a036d71bace4af57c84fa
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
error: could not apply fbd46ff... main: ajusta la linea 2
hint: After resolving the conflicts, mark them with
hint: "git add/rm <pathspec>", then run
hint: "git cherry-pick --continue".
hint: You can instead skip this commit with "git cherry-pick --skip".
hint: To abort and get back to the state before "git cherry-pick",
hint: run "git cherry-pick --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
[exit 1]

$ git status
On branch feature
You are currently cherry-picking commit fbd46ff.
  (fix conflicts and run "git cherry-pick --continue")
```

Captura 2 de 5 comando y salida reales

GITCAP07 - cherry-pick

```
On branch feature
You are currently cherry-picking commit fbd46ff.
  (fix conflicts and run "git cherry-pick --continue")
  (use "git cherry-pick --skip" to skip this patch)
  (use "git cherry-pick --abort" to cancel the cherry-pick operation)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   app.txt

no changes added to commit (use "git add" and/or "git commit -a")

$ git show :1:app.txt
linea 1
linea 2
linea 3

$ git show :2:app.txt
linea 1
linea 2 (feature)
linea 3

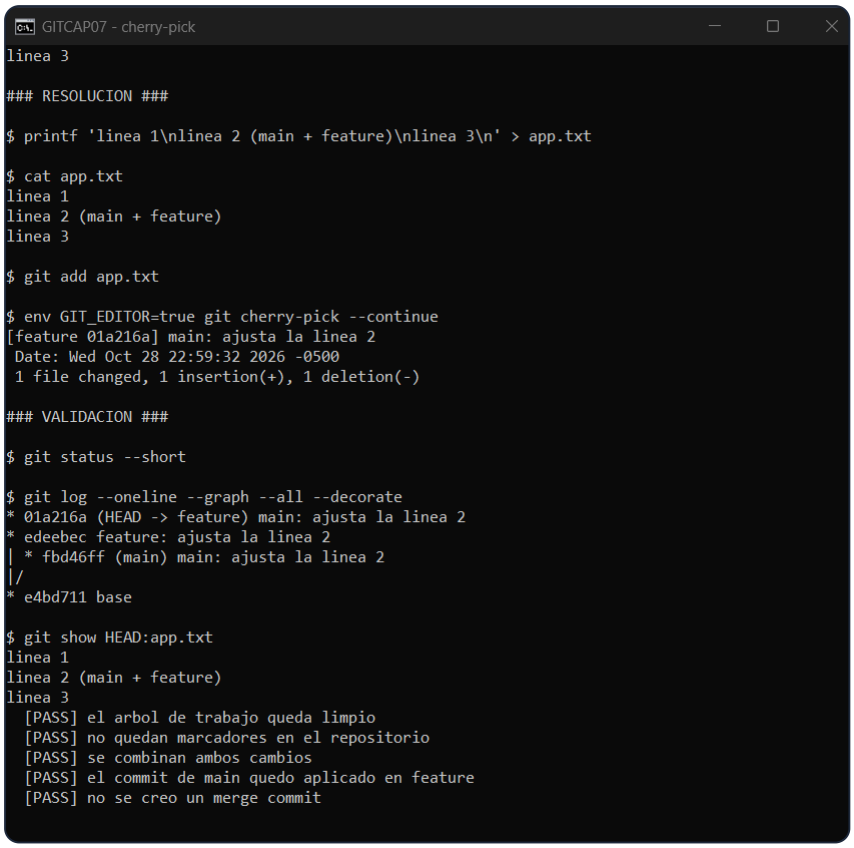
$ git show :3:app.txt
linea 1
linea 2 (main)
linea 3

$ cat app.txt
linea 1
<<<<<<< HEAD
linea 2 (feature)
||||||| parent of fbd46ff (main: ajusta la linea 2)
linea 2
=======
linea 2 (main)
>>>>>>> fbd46ff (main: ajusta la linea 2)
linea 3

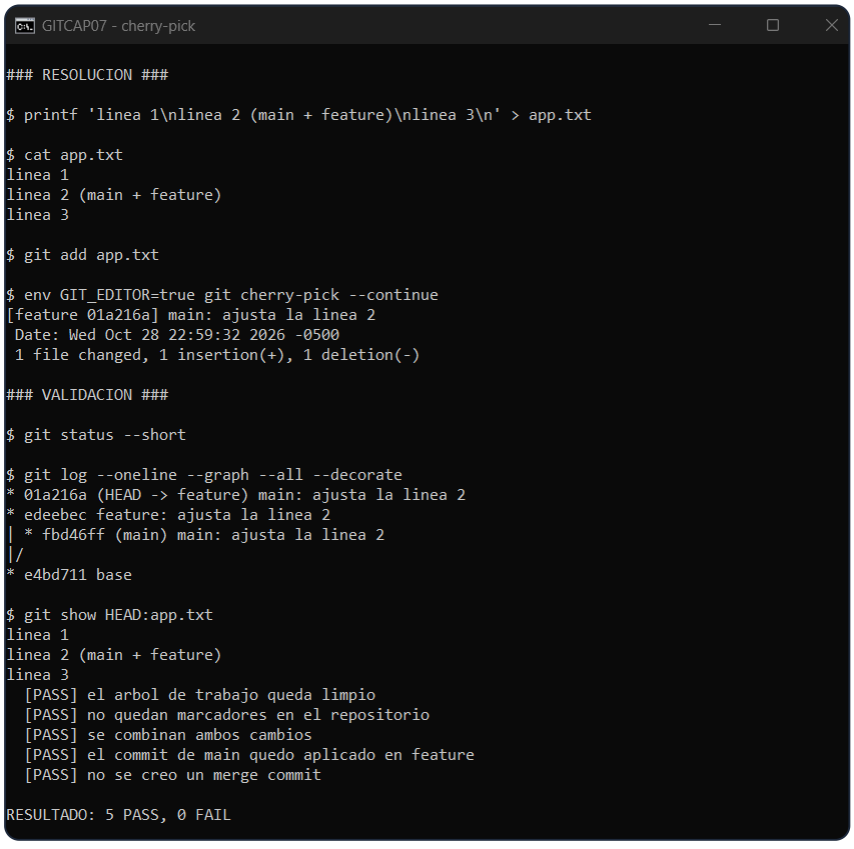
### RESOLUCION ###
```

Captura 3 de 5 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 y 5 de 5 completan la secuencia del ejercicio 07.



Captura 4 de 5 comando y salida reales



Captura 5 de 5 comando y salida reales

EJERCICIO 08 / 10 · INTERMEDIA

STASH POP

Conflicto al recuperar un stash

QUÉ ESTÁ PASANDO

Guardaste trabajo a medias con `stash`, la rama principal avanzó y al recuperarlo choca con el commit nuevo. Verás **Updated upstream** y **Stashed changes**.

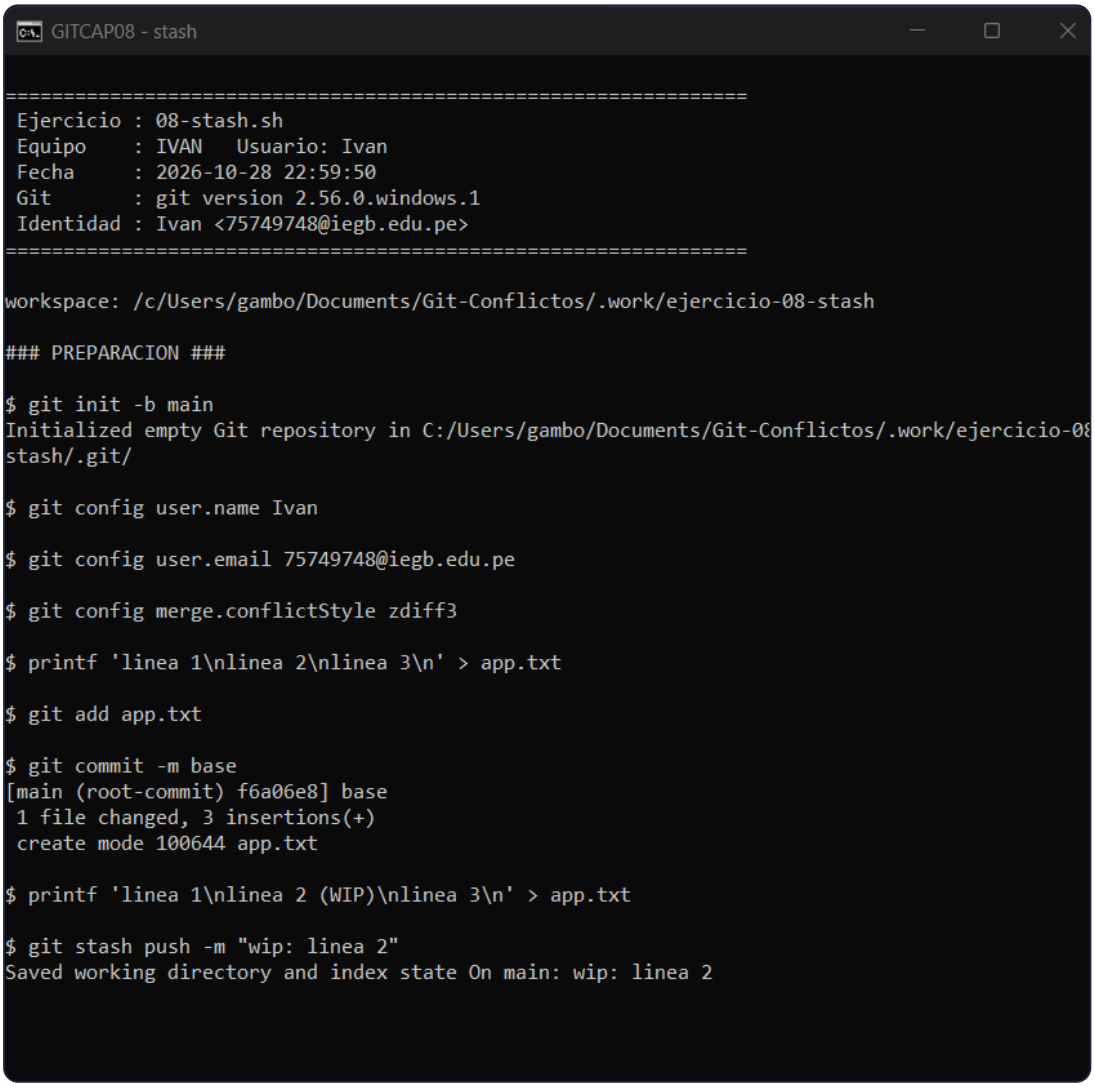
CÓMO SE RESUELVE

1. Recupera el trabajo guardado y observa el conflicto.
   `git stash pop`

2. Combina tu WIP con el cambio de main y elimina las marcas.
   `app.txt`

3. Marca la resolución, elimina el stash ya aplicado y confirma.
   `git add + stash drop`

4 PASS | **0** FAIL — Árbol limpio. El WIP se recuperó y el stash quedó vacío.



Captura real 1 de 4 · consola del equipo — 4 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 4 completan la secuencia del ejercicio 08.

GITCAP08 - stash

```
$ git stash push -m "wip: linea 2"
Saved working directory and index state On main: wip: linea 2

$ git stash list
stash@{0}: On main: wip: linea 2

$ printf 'linea 1\nlinea 2 (commit)\nlinea 3\n' > app.txt

$ git commit -am "main: cambia la linea 2"
[main a67ed94] main: cambia la linea 2
 1 file changed, 1 insertion(+), 1 deletion(-)

### CONFLICTO ###

$ git stash pop
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
On branch main
Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
        both modified:   app.txt

no changes added to commit (use "git add" and/or "git commit -a")
The stash entry is kept in case you need it again.
[exit 1]

$ git status --short
UU app.txt

$ git stash list
stash@{0}: On main: wip: linea 2

$ cat app.txt
linea 1
<<<<<<< Updated upstream
linea 2 (commit)
||||||| Stash base
```

Captura 2 de 4 comando y salida reales

GITCAP08 - stash

```
<<<<<<< Updated upstream
linea 2 (commit)
||||||| Stash base
linea 2
=======
linea 2 (WIP)
>>>>>>> Stashed changes
linea 3

### RESOLUCION ###

$ printf 'linea 1\nlinea 2 (commit + WIP)\nlinea 3\n' > app.txt

$ cat app.txt
linea 1
linea 2 (commit + WIP)
linea 3

$ git add app.txt

$ git stash drop
Dropped refs/stash@{0} (0df6643730718240993489 9e55bc78ae560f582b)

$ git commit -m "feat: recupera el WIP combinado con el cambio de main"
[main 245c346] feat: recupera el WIP combinado con el cambio de main
 1 file changed, 1 insertion(+), 1 deletion(-)

### VALIDACION ###

$ git status --short

$ git stash list

$ git show HEAD:app.txt
linea 1
linea 2 (commit + WIP)
linea 3

$ git log --oneline --graph --all --decorate
```

Captura 3 de 4 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 de 4 completan la secuencia del ejercicio 08.

GITCAP08 - stash

```
### RESOLUCION ###

$ printf 'linea 1\nlinea 2 (commit + WIP)\nlinea 3\n' > app.txt

$ cat app.txt
linea 1
linea 2 (commit + WIP)
linea 3

$ git add app.txt

$ git stash drop
Dropped refs/stash@{0} (0df6643730718240993489 9e55bc78ae560f582b)

$ git commit -m "feat: recupera el WIP combinado con el cambio de main"
[main 245c346] feat: recupera el WIP combinado con el cambio de main
 1 file changed, 1 insertion(+), 1 deletion(-)

### VALIDACION ###

$ git status --short

$ git stash list

$ git show HEAD:app.txt
linea 1
linea 2 (commit + WIP)
linea 3

$ git log --oneline --graph --all --decorate
* 245c346 (HEAD -> main) feat: recupera el WIP combinado con el cambio de main
* a67ed94 main: cambia la linea 2
* f6a06e8 base
  [PASS] el arbol de trabajo queda limpio
  [PASS] no quedan marcadores en el repositorio
  [PASS] se combinan commit y WIP
  [PASS] el stash se elimino despues de resolver

RESULTADO: 4 PASS, 0 FAIL
```

Captura 4 de 4 comando y salida reales

EJERCICIO 09 / 10 · INTERMEDIA

MERGETOOL

Inspección y herramienta de merge

QUÉ ESTÁ PASANDO

Antes de editar conviene reconstruir tres versiones: la base (`:1:`), tu lado (`:2:`) y el otro (`:3:`). Con esa información, una mergetool permite elegir el resultado sin adivinar.

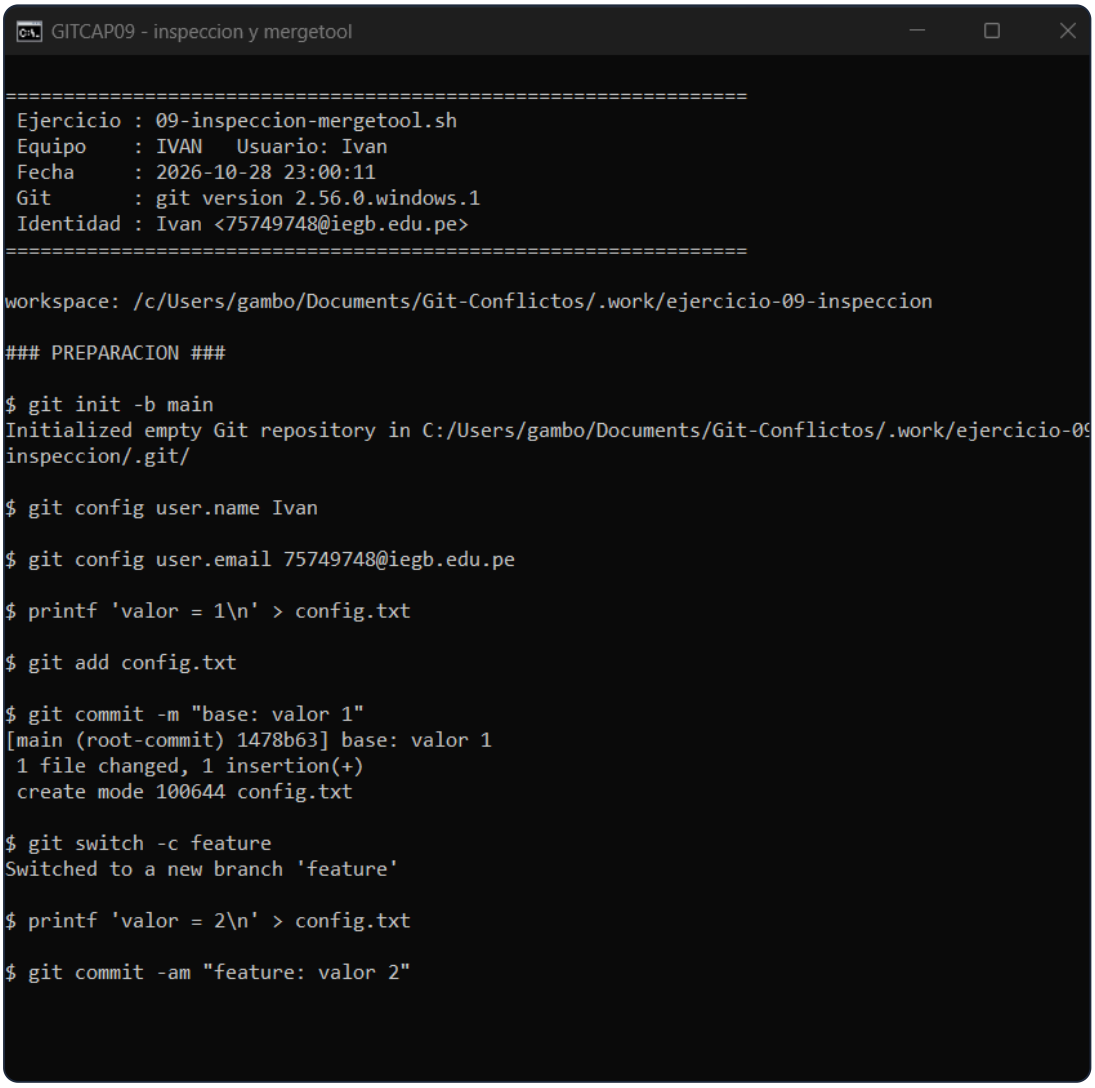
CÓMO SE RESUELVE

1. Lee la base, tu versión y la del otro lado.
   `git show :1: :2: :3:`

2. Abre la herramienta y construye el resultado final de config.txt.
   `git mergetool`

3. Guarda, marca y confirma el merge.
   `git add + git commit`

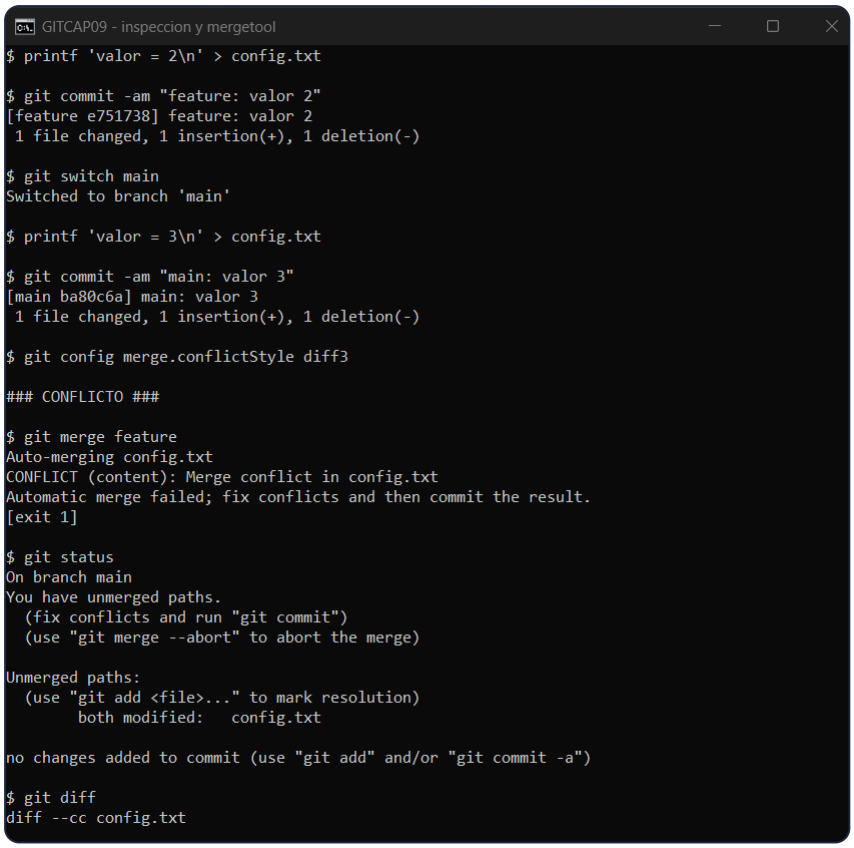
4 PASS | **0** FAIL — Árbol limpio. config.txt se resolvió con información completa.



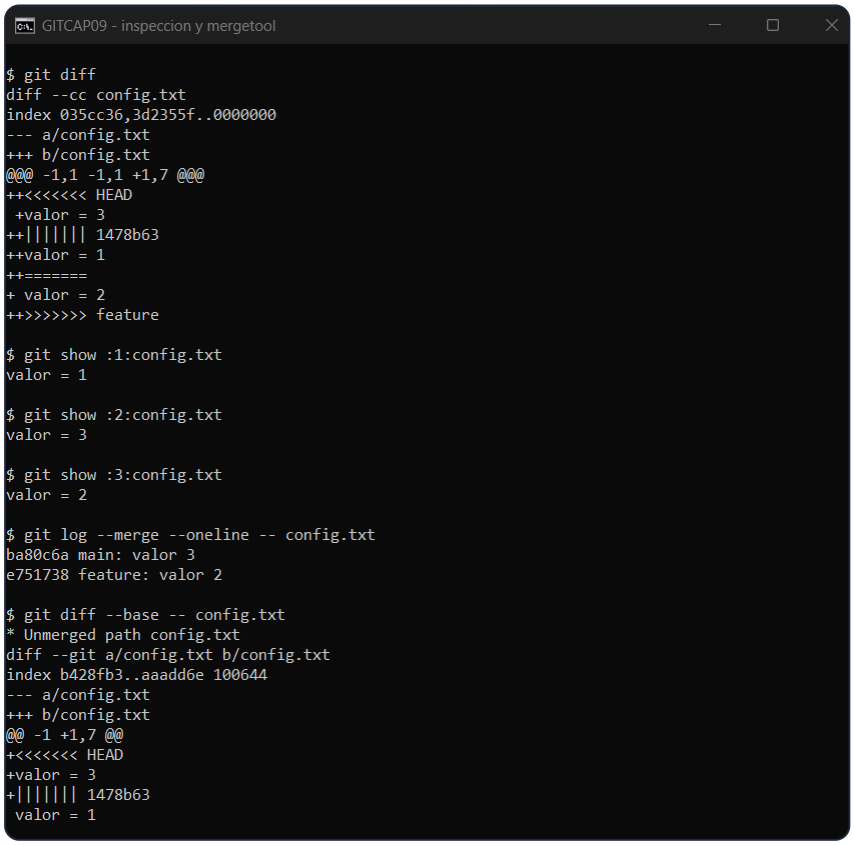
Captura real 1 de 6 · consola del equipo

4 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 6 completan la secuencia del ejercicio 09.

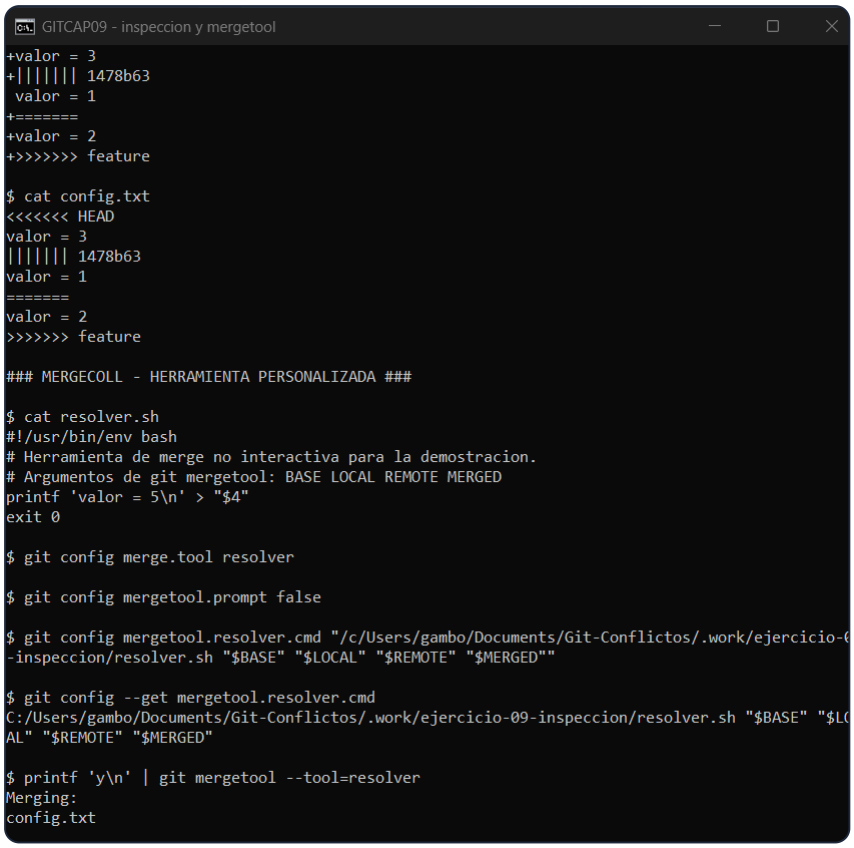


Captura 2 de 6 comando y salida reales

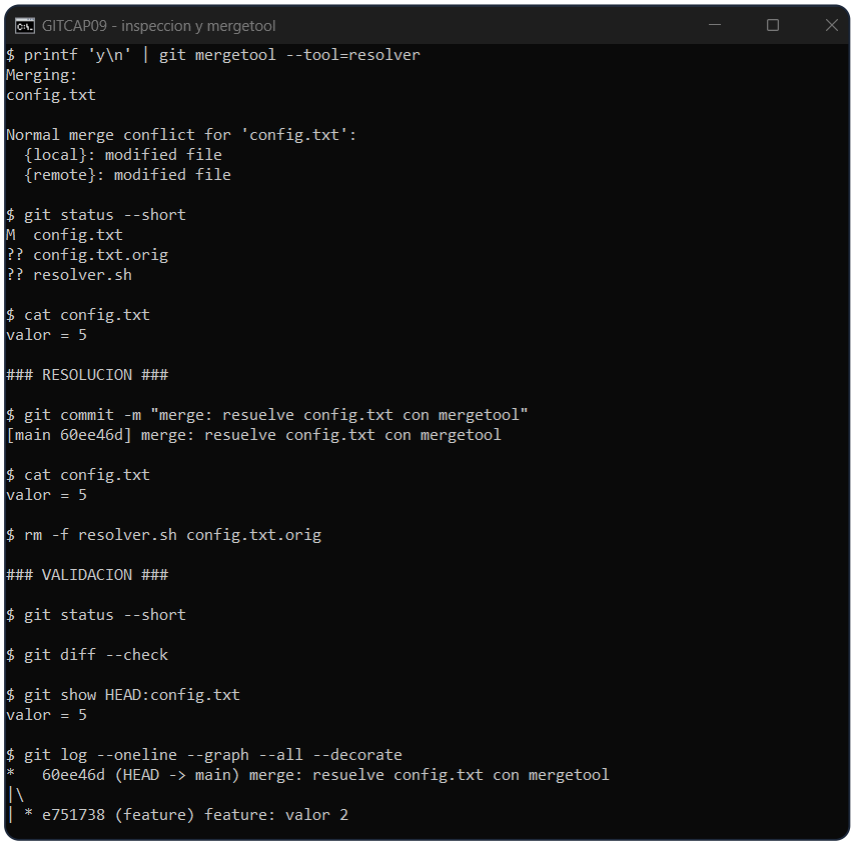


Captura 3 de 6 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 y 5 de 6 completan la secuencia del ejercicio 09.

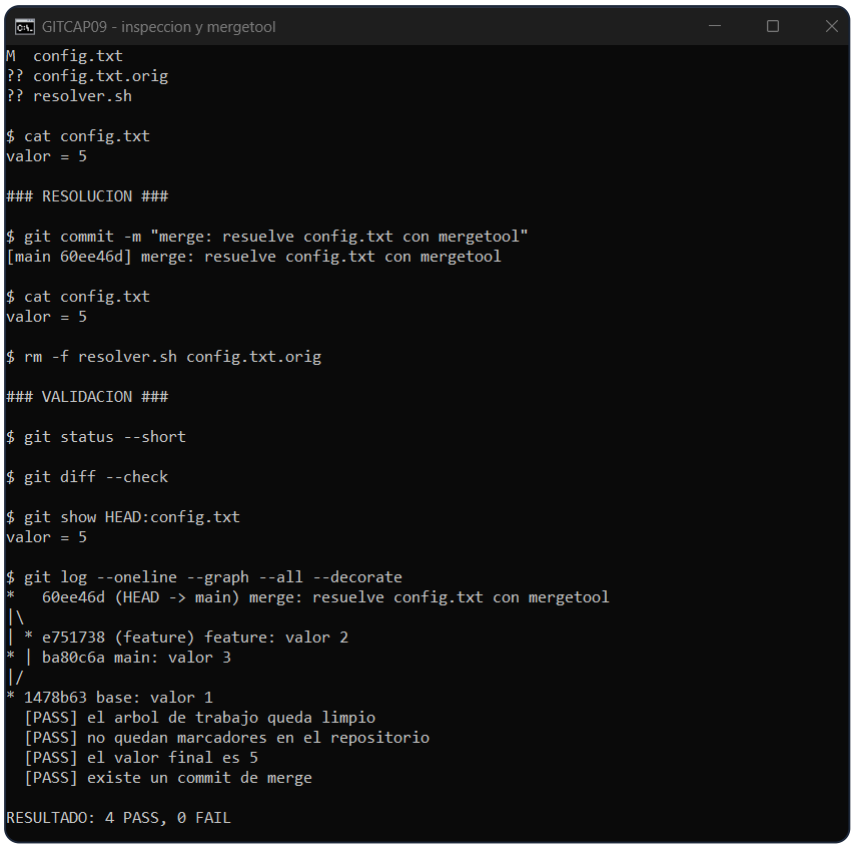


Captura 4 de 6 comando y salida reales



Captura 5 de 6 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 6 de 6 completan la secuencia del ejercicio 09.



Captura 6 de 6 comando y salida reales

EJERCICIO 10 / 10 · AVANZADA

ESTRATEGIAS

Estrategias automáticas

QUÉ ESTÁ PASANDO

Git ofrece atajos para casos claros: `-X ours`, `-X theirs`, un driver `merge=ours` y la estrategia `union`. Sirven para archivos previsibles, pero no sustituyen la revisión.

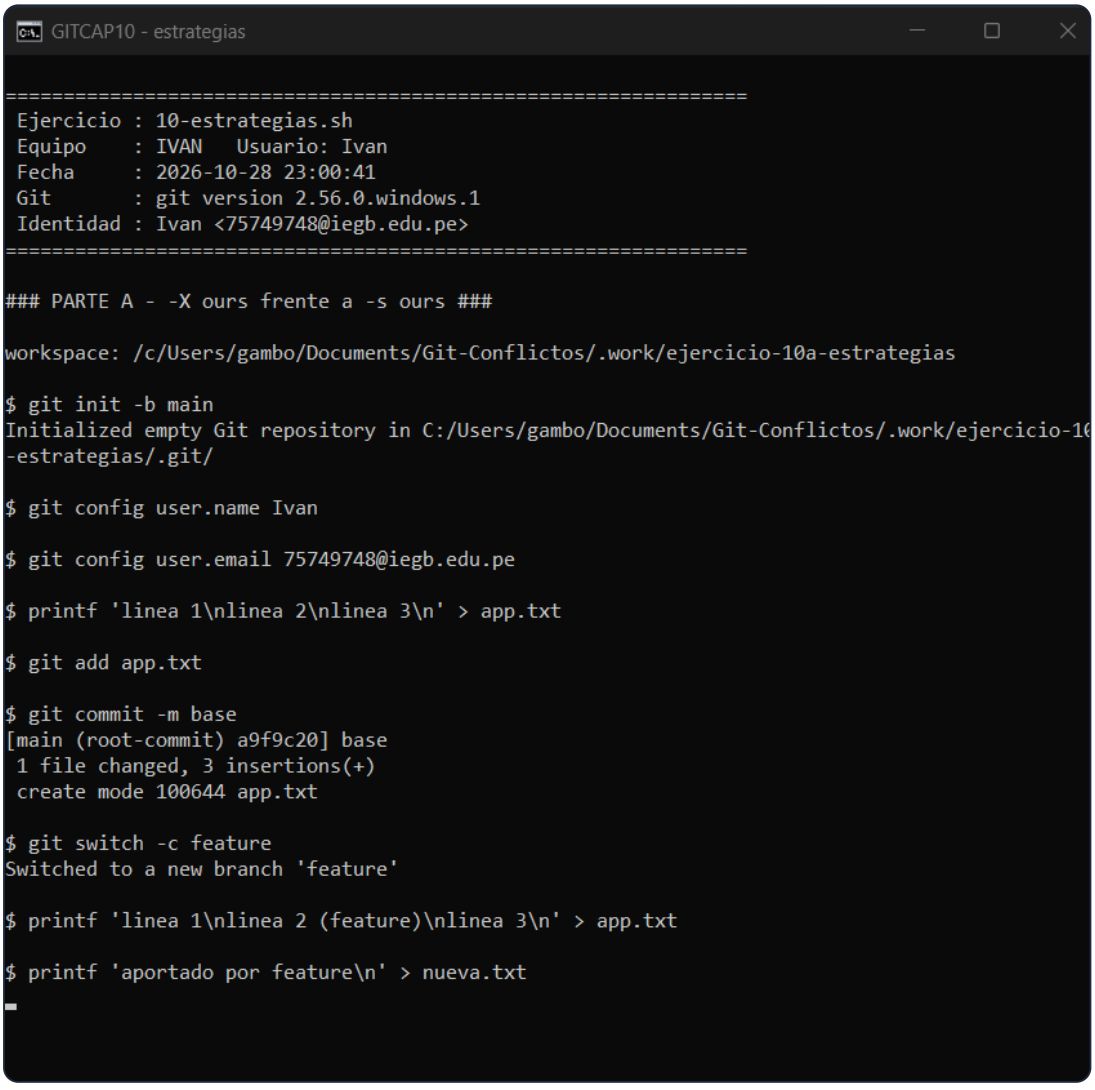
CÓMO SE RESUELVE

1. Prueba las dos estrategias en ramas desechables y comprueba qué se conserva.
   `-X ours · -X theirs`

2. Configura un driver para archivos de lock que no deben cambiar.
   `merge=ours`

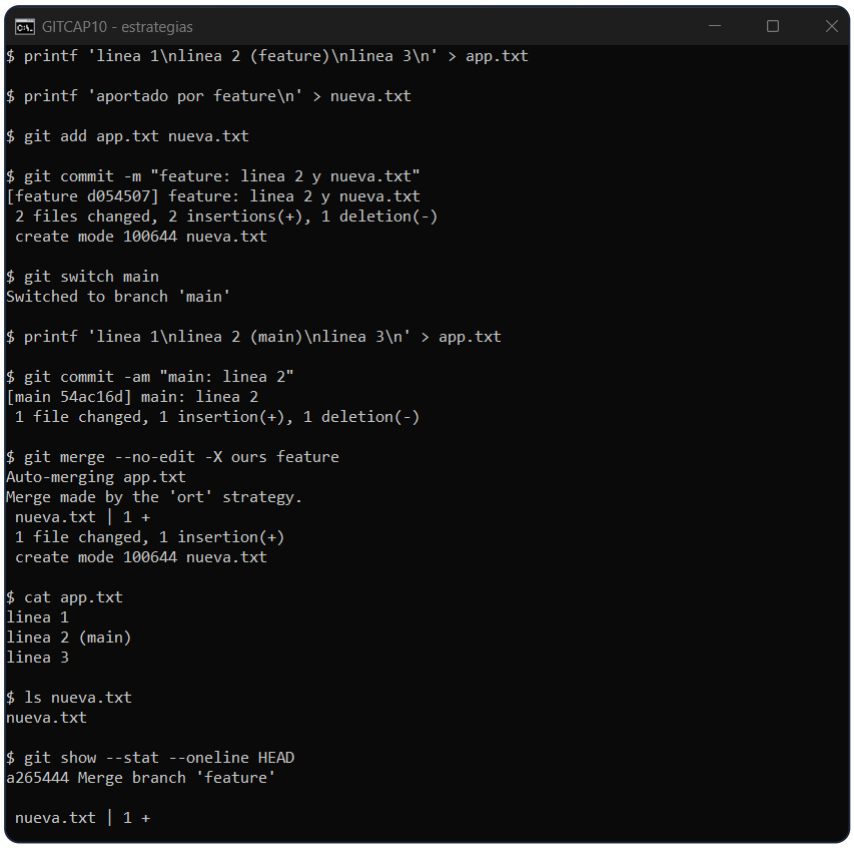
3. Conserva las dos líneas cuando ambas aportan información, y valida el resultado.
   `union`

15 PASS | **0** FAIL — Árbol limpio. Cada estrategia hizo exactamente lo esperado.

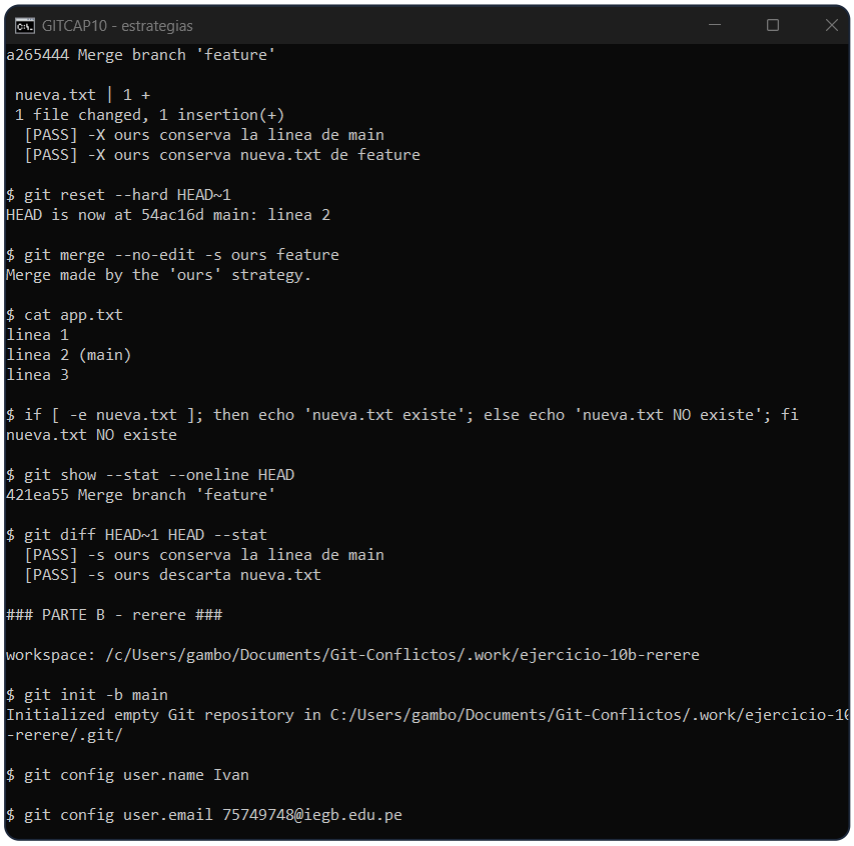


Captura real 1 de 10 · consola del equipo — 15 PASS · 0 FAIL

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 2 y 3 de 10 completan la secuencia del ejercicio 10.

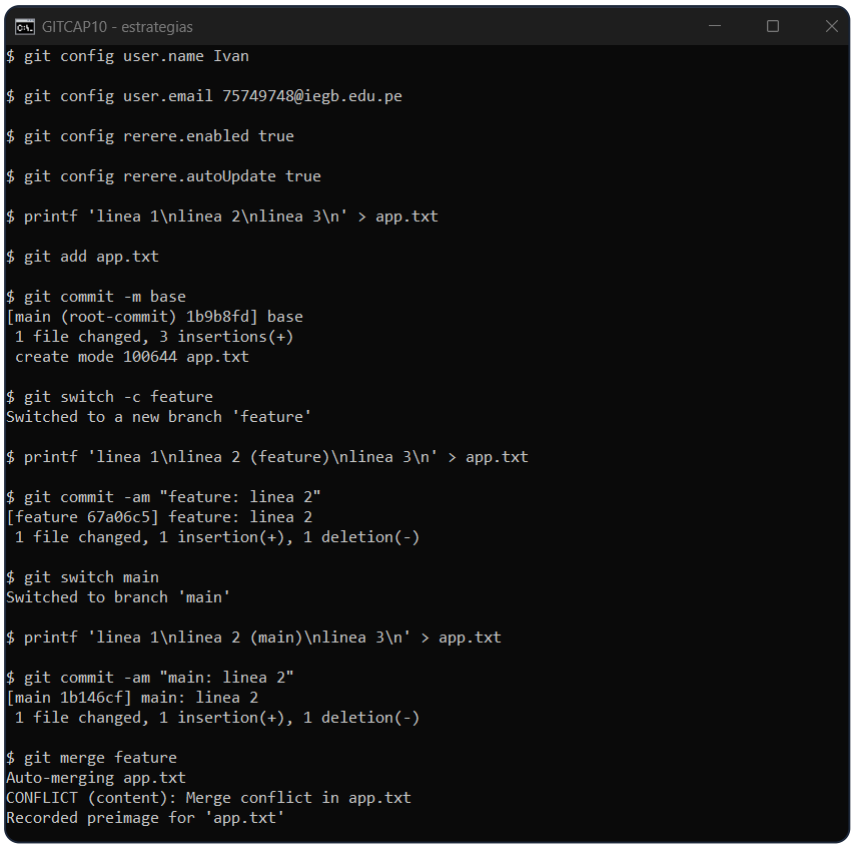


Captura 2 de 10 comando y salida reales

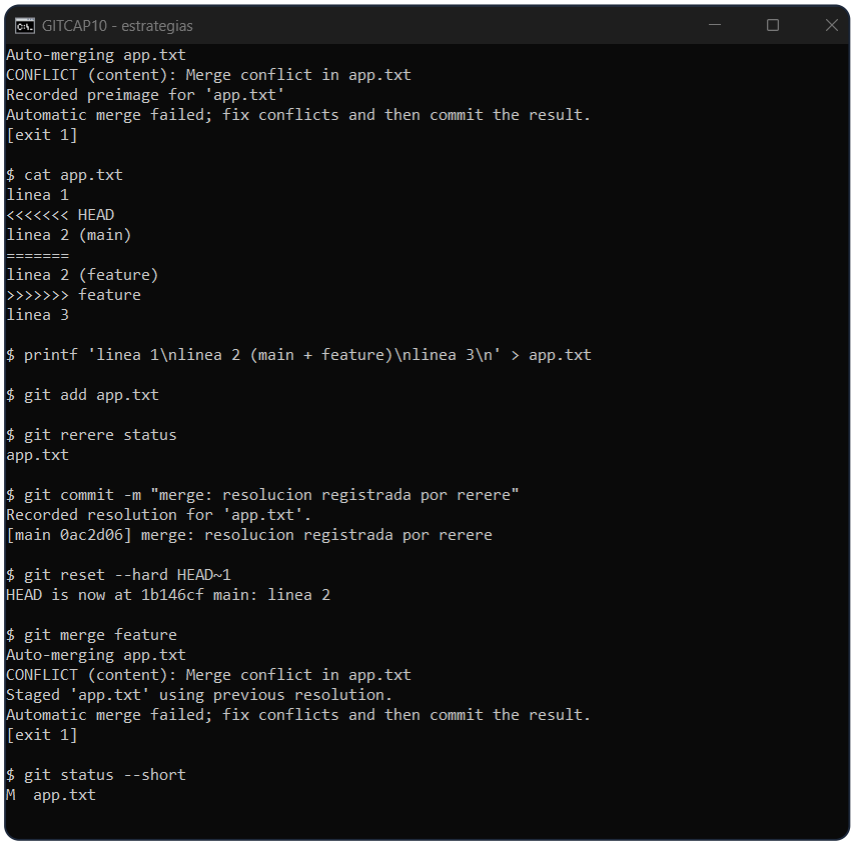


Captura 3 de 10 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 4 y 5 de 10 completan la secuencia del ejercicio 10.

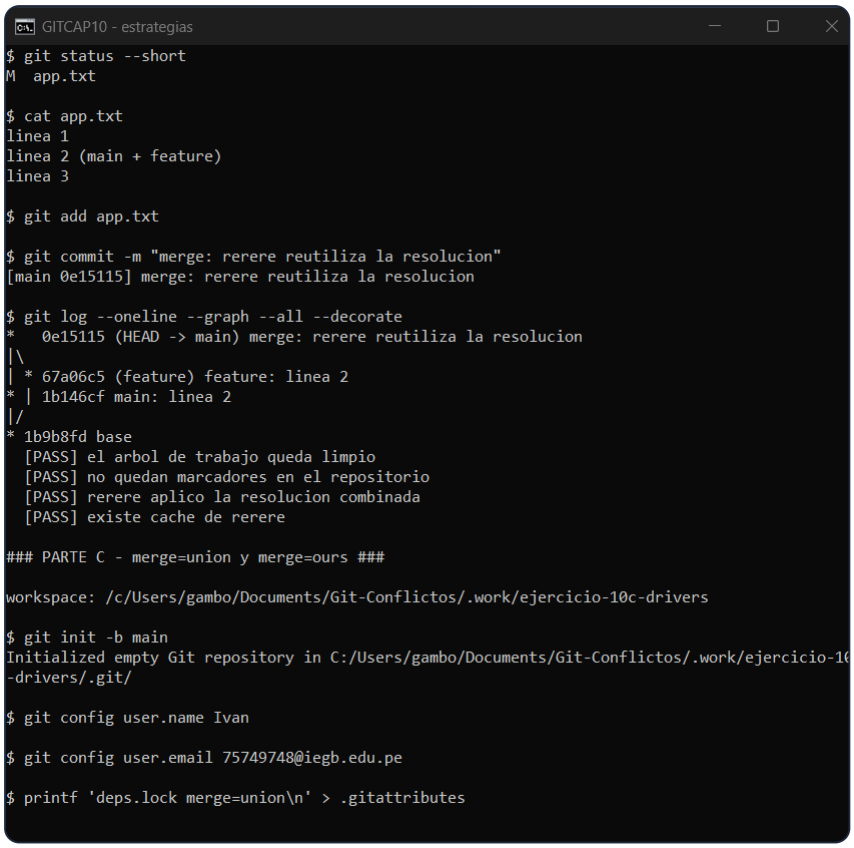


Captura 4 de 10 comando y salida reales

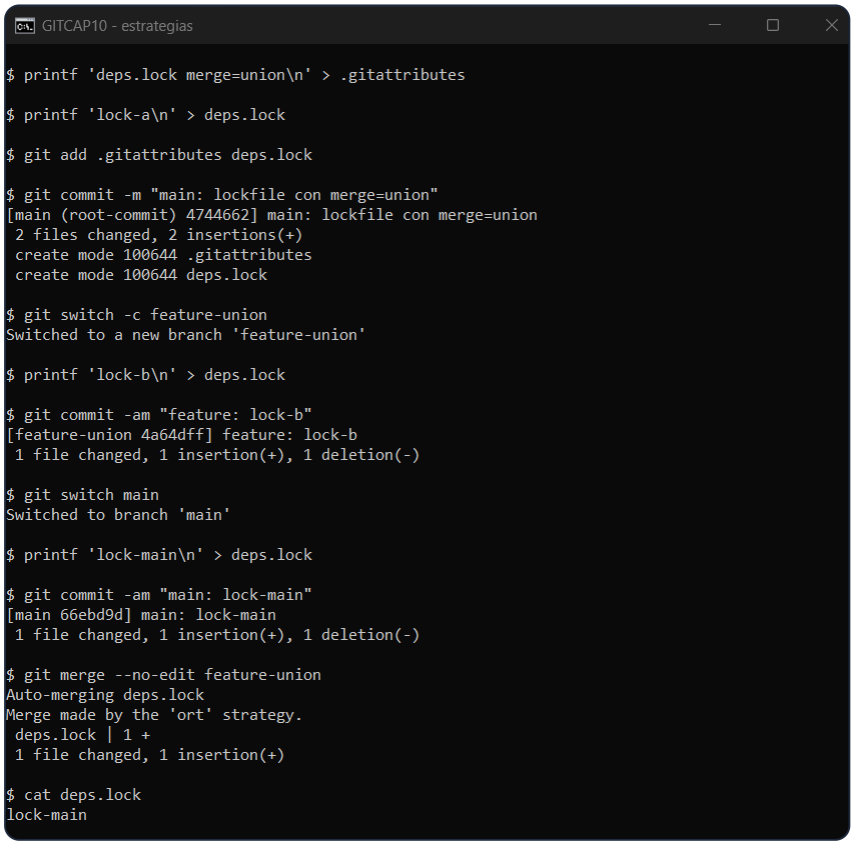


Captura 5 de 10 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 6 y 7 de 10 completan la secuencia del ejercicio 10.

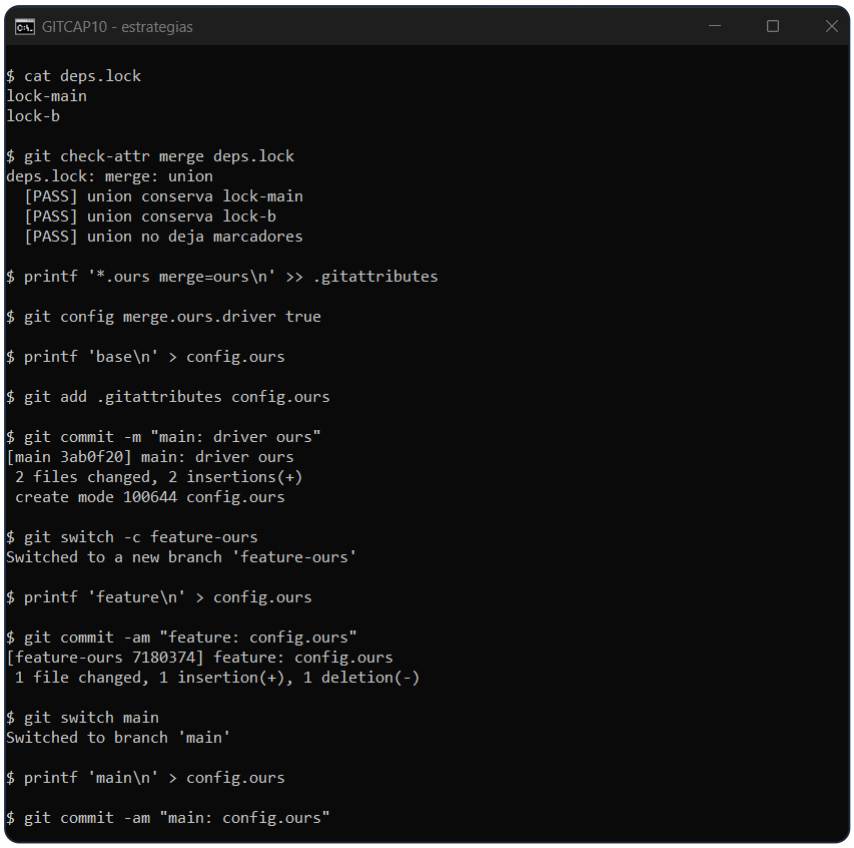


Captura 6 de 10 comando y salida reales

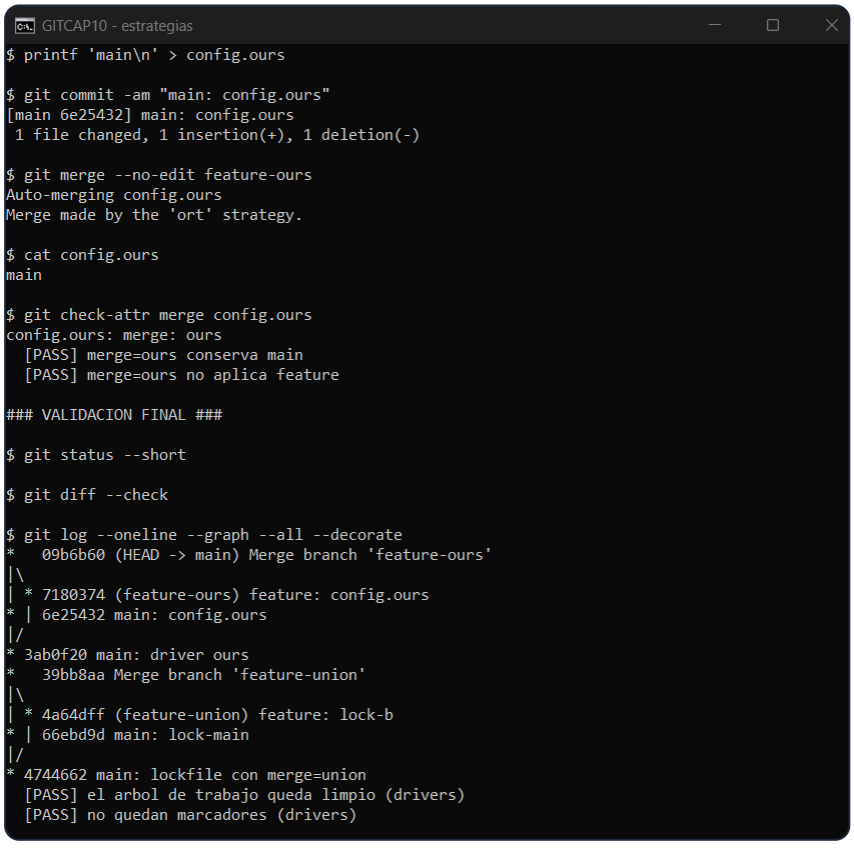


Captura 7 de 10 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 8 y 9 de 10 completan la secuencia del ejercicio 10.

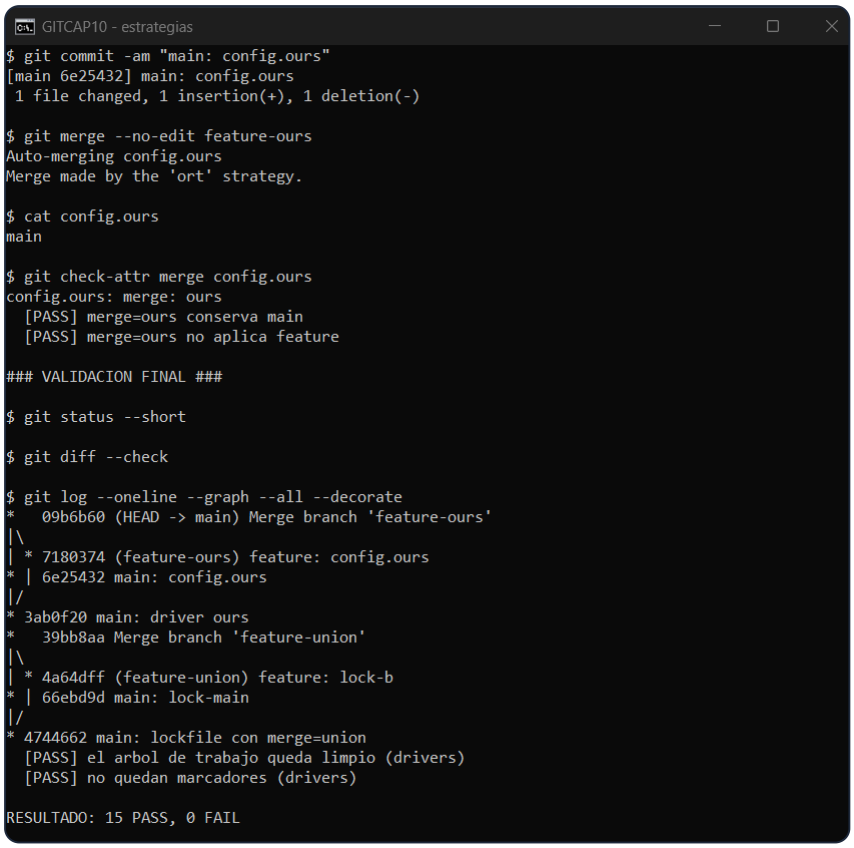


Captura 8 de 10 comando y salida reales



Captura 9 de 10 comando y salida reales

Continuación de la consola real. Cada línea `$` es un comando escrito en este equipo y debajo aparece su salida. Las capturas 10 de 10 completan la secuencia del ejercicio 10.



Captura 10 de 10 comando y salida reales

RESULTADOS

10 ejercicios, 57 comprobaciones

Cada número sale de la última línea del log real: RESULTADO: N PASS, 0 FAIL.

10/10	57	0	51
ejercicios completados	comprobaciones PASS	fallos	capturas reales

#	Ejercicio	PASS	FAIL	Estado
01	Conflicto simple en una línea	5	0	✓ Verificado
02	Varios conflictos en un archivo	6	0	✓ Verificado
03	Conflicto modificar / eliminar	4	0	✓ Verificado
04	Conflicto añadir / añadir	5	0	✓ Verificado
05	Conflicto con un archivo binario	4	0	✓ Verificado
06	Conflicto durante un rebase	5	0	✓ Verificado
07	Conflicto en un cherry-pick	5	0	✓ Verificado
08	Conflicto al recuperar un stash	4	0	✓ Verificado
09	Inspección y herramienta de merge	4	0	✓ Verificado
10	Estrategias automáticas	15	0	✓ Verificado

Qué significa "verificado". El script creó el conflicto, aplicó la resolución y ejecutó comprobaciones automáticas. El árbol de trabajo quedó limpio, sin marcadores, en los diez casos.

OPERACIONES CUBIERTAS

merge · rebase · cherry-pick · stash pop · binarios · mergetool · estrategias

CÓMO SE GENERÓ LA EVIDENCIA

1. **Se crea el conflicto**
   Un script prepara un repositorio temporal y provoca el choque real.

2. **Se resuelve y valida**
   El ejercicio aplica la solución y ejecuta comprobaciones PASS/FAIL.

3. **Se guarda la prueba**
   Las capturas de consola y el log completo quedan en el repositorio.

PARA CERRAR

Tu chuleta de bolsillo

Lo que conviene recordar cuando la terminal se pone en rojo.

Si te bloqueas

1. No borres el archivo ni la carpeta `.git`.
2. Ejecuta `git status --short` y mira qué archivos están en conflicto.
3. Ejecuta `git diff` y lee las dos versiones antes de editar.
4. Si te pierdes, aborta: `git merge --abort`, `git rebase --abort` o `git cherry-pick --abort`.
5. Vuelve a empezar con calma. Abortar también es una solución válida.

Nunca hagas esto

- ✗ Commitear un archivo que todavía contiene `<<<<<<<`.
- ✗ Usar `-X ours` o `-X theirs` sin leer los cambios.
- ✗ Borrar `.git` para "arreglar" un conflicto.
- ✗ Resolver a ciegas solo para que el comando termine.

LA PREGUNTA CORRECTA

¿Qué quería hacer cada lado?

DÓNDE ESTÁ TODO

REPOSITORIO PÚBLICO

github.com/75749748-blip/conflictos-git-ejercicios

`ejercicios/` scripts y logs · `build/real-captures/` capturas de consola · `mapa-mental/` canvas HTML5 y PNG · `documentacion/` esta guía

CHECKLIST ANTES DE CONFIRMAR

- ✓ No quedan marcas `<<<<<<<`
- ✓ `git diff --check` no muestra errores
- ✓ Las pruebas del proyecto pasan
- ✓ Entiendo qué cambió cada lado

Un conflicto bien resuelto deja el historial más claro, no más sucio.

10/10 ejercicios · 57 PASS · 0 FAIL